数据中心的“税”

RPC 与微服务负载洞察，及其对软硬件发力点的含义

通用负载长什么样 → CPU 周期花在哪 → 学界怎么削 → 鲲鹏 CPU / KAE / UB 先测什么、先做什么

基于 60 篇论文（2015–2026）的综述，完整版（团队学习 / 评审用）
配套报告 REPORT.md （6 章 + 附录）· 论文笔记 notes/01–43 · 图表均截自论文原文并标注来源

Kunpeng 基础加速 / 洞察工作组 · 2026-09-22

阅读地图

这份综述回答三个问题，并把答案落到“芯片与服务器厂商该在哪里发力”

报告要回答的三个问题（与 PPT 五个部分的对应）

① 数据中心的负载是什么样的，“通用负载”里 RPC 占多大比例?

→ 第一部分（第 1 章）：没有杀手应用、税的构成与十年演化、 RPC 的三个口径、 RPC 的大小 / 时延 / 调用树分布、微架构画像

② 微服务有哪些部署形态， RPC 负载是怎么产生的?

→ 第二部分（第 2 章）：每请求 RPC 数、 RPC 栈跑在哪（ library / sidecar / 节点服务 / 卸载）的成本系数、放置与拓扑、 VM / 容器 / serverless 、三角验证

③ RPC 有哪些研究方向——通信和序列化各自在解决什么?

→ 第三部分（第 3 、 4 章）：内核网络栈的钱花在哪、传输协议 / 内核旁路 / 调度 / 硬件卸载 / CXL 五条路线；序列化成本结构与加速器放置之争

贯穿全文的目标： CPU 周期花在哪、哪些能被更好的软硬件消掉?

→ 第四部分（第 5 章）热点地图与发力点 A / B / C / D ；第五部分（第 6 章）投影到鲲鹏 CPU 、 KAE 、 UB ，写成“文献要什么 → 现状 → 差距 → 怎么验证”

论文覆盖（ 60 篇，其中 2024–2026 的近期文献 20 余篇）

论文类别	篇数	年份
负载刻画 / benchmark （ Google 、 Meta 、阿里、 Uber ）	16	2015–2025
微服务部署形态（ service mesh 、 RPC-as-service 、 FaaS ）	10	2019–2026
RPC 通信栈（传输、内核旁路、调度、硬件卸载、 CXL ）	20	2018–2025
RPC 序列化与压缩（软件、零拷贝、加速器）	11	2020–2026
面向微服务的 CPU / SKU 设计	3	2019–2024

数据来源分三层：三家 hyperscaler （ Google / Meta / 阿里）的生产测量 → 学术 benchmark （ DeathStarBench 等）→ 方案论文的原型评估。三层数字相差可达一个数量级，本 PPT 逐页标口径。

文中 [NN]= notes/ 目录下的笔记编号，可回查原文数据。

主线：负载数据（第一、二部分）决定“税”落在哪、多大 → 研究方向（第三部分）告诉我们能削多少 → 发力点与鲲鹏映射（第四、五部分）是行动项。

术语速查：后面每一页都会用到的十个词

Datacenter tax （数据中心税）

跨应用共享、与业务逻辑无关的底层例程耗掉的 CPU 周期： RPC 、序列化、压缩、内存分配、拷贝、哈希等。 Google 2015 年命名 [01] 。

fleet （机群） / fleet 平均

一家公司全部服务器的总体。“ fleet 平均 x%”指全部 CPU 周期里的 x% ；“某热点服务里 y%”是另一个口径，两者常差一个数量级。

RPC （远程过程调用）

服务 A 像调本地函数一样调服务 B 。一次 RPC 经过：序列化请求 → 网络栈 → 网线 → 对端排队 → 处理 → 序列化响应 → 回程。

序列化 / 反序列化

把内存对象变成可传输的字节流 / 再变回来（ protobuf 、 Thrift 、 FlatBuffers ）。注意：不只发生在 RPC 上，存储读写、日志、进程内传递也用。

微服务 / 单体

把一个应用拆成很多小服务、靠 RPC 通信 / 整个应用在一个进程里。拆分越细，每请求 RPC 数越多。

部署形态 library / sidecar

RPC 栈编进应用进程（ library ， Google Stubby 、 Meta SRLib ） vs 每个应用旁挂一个代理进程（ sidecar ， Envoy / Istio ）。

P50 / P99 / 尾延迟

一批请求的延迟从小到大排序，中位数 / 第 99 百分位；“尾延迟”指最慢的那一小部分请求，通常由排队和干扰决定。

IPC / MPKI / 前端 stall

每周期完成的指令数 / 每千条指令的 cache 未命中次数 / 流水线因取不到指令而空转的比例（ Top-Down 方法的 front-end bound ）。

内核旁路 / 零拷贝

应用绕过操作系统内核直接操作网卡（ DPDK 、 RDMA ） / 数据从应用到网卡不在内存里多搬一遍。

卸载（ offload ）

把某段工作从 CPU 核搬到网卡、 DPU 、片上加速器等专用硬件上做；关键变量是“放在哪”和“调用一次的固定开销”。

数据来源与口径：术语定义综合 [01][02][23] 与本报告第 0 章；括号内为代表性系统或论文。

骨架公式

全篇的骨架是一个乘法公式：机群级 RPC 开销 = 请求量 × 每请求 RPC 数 × 每 RPC 成本

fleet 级 RPC 开销 ≈ Σ 服务 [请求量 × 每请求 RPC 数 × 每 RPC 成本]

每 RPC 成本 = 通信 + 序列化 + 排队 / 调度 + 压缩 / 加密，再乘上部署形态系数（RPC 栈跑在哪、跑在什么隔离环境里）

请求量

谁决定：业务本身，不在本报告范围。

但增速很快： Google 每 CPU 周期承载的 RPS 每年 +30%（700 天 +64%）—— RPC 增长快于算力 [02]。

含义：即使每 RPC 成本不变，RPC 税在 fleet 里的绝对量也在涨。

×

每请求 RPC 数

谁决定：微服务拆分粒度 + 调用图形状（宽 / 深）+ 放置拓扑（部署形态 C）。

生产数据： Meta 一次 Fetch 请求中位 498 次调用、P99 宽 7,400 [03]； Google 半数方法中位后代 ⩽ 13 [02]。

对应：第二部分 2.1；软件手段是合并接口、同进程多组件、CXL 共享内存。

×

每 RPC 成本 × 部署形态系数

谁决定： RPC 栈跑在哪（形态 A：进程内 library / sidecar / 节点级服务 / 内核旁路 / 硬件卸载）、跑在什么隔离里（形态 B： VM / 容器 / serverless）。

生产数据： Meta 生产 library +36%、sidecar 代理 +273% CPU [21]； serverless + mesh 策略 p50 慢 7× [43]。

对应：第二部分 2.2–2.4；削减方案：通信 = 第 3 章，序列化 / 压缩 = 第 4 章。

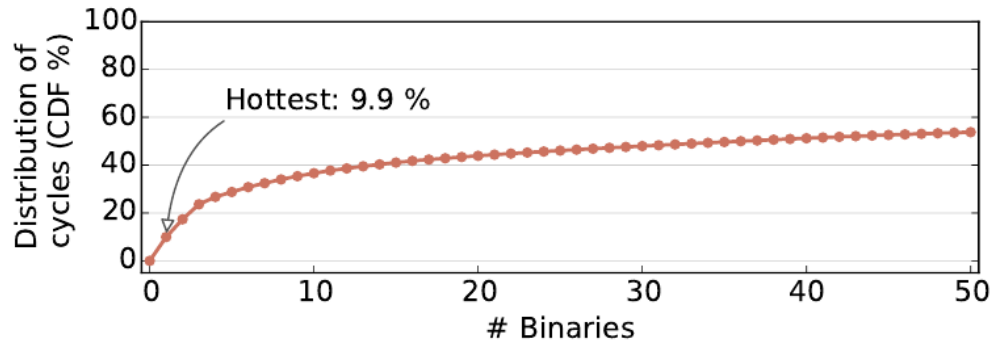
第一部分给出公式左边（测量结果）和右边各项的 fleet 级数值；第二部分解释两个乘数怎么产生并做三角验证；第三部分的每篇论文都在削某一项；第四部分按“落在哪块硅 × 量级 × 能削多少”画热点地图。

数据来源与口径：公式为本报告的组织方式（REPORT.md 0.2 节），非某篇论文原文；引用数字见对应页。

数据中心的负载长什么样

没有杀手应用 → “税”的构成与十年演化 → RPC 占比的三个口径 → RPC 的大小 / 时延 / 调用树分布 → 微架构画像： CPU 的钱花在哪

没有杀手应用：最热的单个程序只占机群 CPU 周期的约 10%，前 50 个合计约 60% 且逐年下降



Google 机群 CPU 周期按二进制累计分布：最热者 9.9%，前 50 个约 60%（[01] Fig. 1）

≈10%

最热的单个二进制占 fleet 周期

Google 2015，≈ 2 万台机器 [01]

≈60%

前 50 个二进制合计

且每年下降约 5 个百分点 [01]

6.3%

搜索叶子节点最热的函数

353 个函数才覆盖 80% 周期 [01]

≈50%

malloc 周期中 top-50 二进制覆盖率

Google 2024 [35]

负载极度分散，而且越来越分散。从 Google 2015 的二进制分布到 2024 的 malloc 归因，“头部程序覆盖率”始终只有一半左右，而且每年下滑。

应用内部也没有热点。搜索叶子节点最热函数只占 6.3%，要 353 个函数才覆盖 80% 的周期——逐应用做微架构优化的收益上限很低。

因此硬件专用化的唯一目标是跨应用共享的底层例程，也就是“税”：RPC、序列化、压缩、内存分配、拷贝、哈希。它们在每个程序里都不大，但加起来是机群里最大的可寻址块。

结论：为单个应用优化的硬件没有 fleet 级价值；只有跨应用共享的“税”才值得硬件专用化——这是全篇的出发点。

数据来源与口径：[01] Kanev 等，ISCA'15，Google ≈2 万台 Ivy Bridge 机器、3 年 GWP 采样，按二进制归因全部 CPU 周期（fleet 平均口径）；[35] Google ASPLOS'24，malloc 周期按二进制归因。

三家 hyperscaler 的负载分类与运行形态：都是“多层服务 + 服务间全走 RPC”，差别在语言和部署方式

Google [01][02][17][25]

服务： Search / Gmail / Maps / YouTube 及其后端 Spanner、BigQuery、Bigtable、F1、GFS。

架构： 自称“面向服务架构而非严格微服务”——服务不拆到最小粒度，单个 RPC 常常计算量大。

RPC： Stubby / gRPC + protobuf，几乎全部走 TCP；RPC 产生 95%+ 的应用流量，其中约 75% 与存储系统交互。

语言： C++ 为主（protobuf 周期 88% 在 C++），另有 Java / Python / Go。

运行： Borg 容器；RPC 栈是进程内 library，其下是用户态 Snap，正演化为 Falcon 硬件传输。

Meta [03][05][20][22]

服务： 通用服务器上数千个服务按功耗归 5 类（DCPerf）：Web（HHVM / PHP、Django）、Ranking（C++）、Data caching（TAO / Memcached）、Big data（Spark / Presto，Java）、Media（视频转码）。

硬件： 非 AI 计算只有一种服务器 SKU（单 CPU + 256 GB DRAM）；存储、AI 服务器另算。

RPC： Thrift 二进制 RPC，16 种语言，按代码量 Hack 最多；99% 的 RPC 走进程内 SRLib，1% 走代理。

运行： 产品代码主要是 serverless 函数（FaaS，>50 万台服务器、>1 万工程师），多个函数在同一 Linux 进程里并发执行以提高硬件效率。

阿里 [04][39]

服务： 在线微服务以 Java 为主；2022 年 64,760 个服务 / 187 万实例，中位每服务 4 个实例。

运行： K8s 容器 + 裸金属，与离线批处理混部；在线容器 CPU 利用率常 <10%；DB / Memcached 独立集群。

通信： 无状态服务间 RPC 76%、消息队列 MQ 23%、IPC 1%——MQ 占比明显高于 Google / Meta。

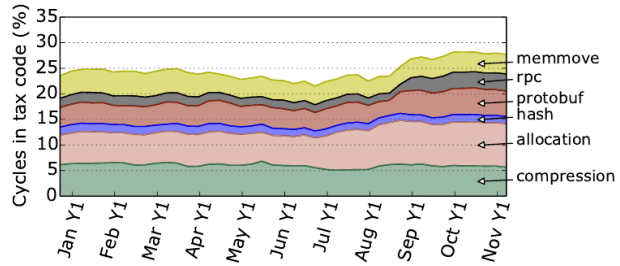
调用图： 5% 的“超级服务”处理 95% 的调用；top 1% 服务 = 87% 调用；常见深度 3。

共性：分布式多层服务，服务间几乎全走 RPC，请求 / 响应用统一格式序列化。差异：语言（C++ / Hack+C++ / Java）与运行形态（library / FaaS 同进程 / 容器混部）——这些差异决定后面每个税项的量级。

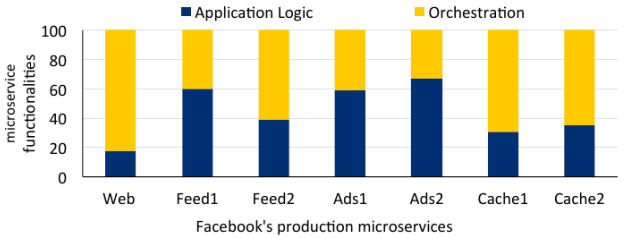
数据来源与口径：[01] Google ISCA'15；[02] Google SOSP'23（1 万个 RPC 方法）；[17] Google SIGCOMM'22 流量口径；[25] Google MICRO'21 protobuf 语言分布；[05] Meta ISCA'25 DCPerf；[20][22] Meta ASPLOS'20 / CACM'25；[04][39] 阿里 SoCC'21 / arXiv'25 生产 trace。

“税”的构成与十年演化： Google 2015 六项合计 22–27%； 2021–24 的分项测量显示分配下降、序列化全口径远大于 RPC 口径

项目	Google 2015 [01]	Google 2021–24	Meta 2020 [20] （7 个重要微服务）	备注
总 tax	22–27% （六项合计；叶子函数归因，偏低估）	—	每微服务 18–82% （应用逻辑最少只有 18%）	Meta 数字是逐服务范围，不是 fleet 平均
内存分配 / 释放	≈6–7%	4.3% [35] （top-5 应用 3.6–10.1%）	内存函数合计最高 37%； free() 意外地贵	98% 分配 <1 KB， 46% 寿命 <1 ms
压缩 / 解压	≈5–6% （≈ 1/4 的 tax）	2.9% [34] （关键服务 10–50%）； RPC 路径上 3.1% [02]	Feed1 15%、 Cache1 15%、 Web 11%	95% 字节用弱算法 / 低级别以省 CPU
memcpy / memmove	4–5% （未含内联）	—	内存函数中的最大项	多数拷贝 <512 B
protobuf / 序列化	≈3–4%	9.6% （全部 protobuf 操作， C++ 占 88%； deser 2.2% + ser 1.25% 可加速）[25]； RPC 路径上 1.2% [02]	Web 10%、 Feed1 13%、 Feed2 9%、 Cache 4%	只有 16% 的反序列化、 35% 的序列化在 RPC 路径上
RPC 库	≈3–4% （1/3 库逻辑 + 2/3 载荷搬运）	1.1% [02]	RPC 收发 I/O： Cache2 52%、 Cache1 38%、 Web 21%	—
TCP / 网络栈	计入内核（≈ 20%）	1.7% [02]	Cache 类约 30% 时间在内核 [05]	—
哈希	≈1–2%	—	—	—
加密	—	<0.2% [02] （连接复用 >99%）	Cache1 6% （叶子函数）	新建 TLS 连接 1.6 ms / 14 KB [21]
日志 / 线程池	—	—	日志： Web 23%；线程池管理 4–11%	日志被学术界忽视
内核（不算可加速税）	≈18–20% （scheduler ≈5%）	—	Cache1/2 上下文切换最多 18% CPU 时间 [36]	内核 IPC 低且跨代不改善



Google 一年内 tax 六项占全部周期 22–27% （[01] Fig. 4）

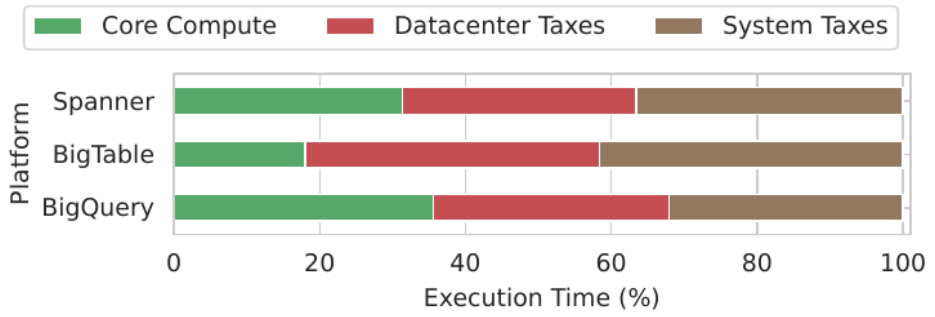


Meta 7 个生产微服务：应用逻辑 vs 编排开销（[20] Fig. 1）

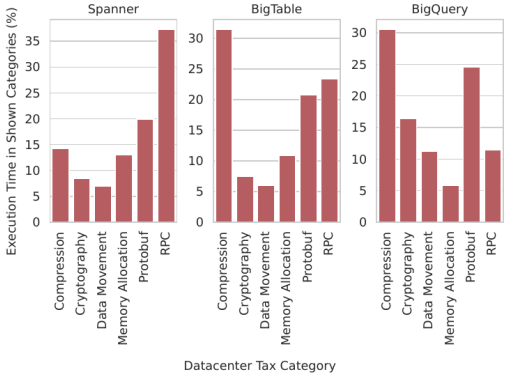
两个可信趋势： (1) 分配 ≈ 6–7% → 4.3% 是 TCMalloc 软件持续优化的结果； (2) 序列化全口径 9.6% ≫ RPC 口径 1.2%—— 大头在存储读写、日志、进程内传递，不在网络 RPC 路径。

数据来源与口径： Google 2015 [01] 为叶子函数归因（偏低估）； Google 2021–24 各项来自不同论文（[35] 分配、 [34] 压缩、 [25] protobuf、 [02] RPC 路径）且口径不同，不能与 2015 直接相减； Meta 2020 [20] 是 7 个重要微服务，不是 fleet 平均。

存储 / 大数据平台是“税”的另一半： Spanner 、 BigTable 、 BigQuery 的 CPU 周期里核心计算只占 18–36%



Google 三个生产平台的周期三分：核心计算 / datacenter tax / system tax （[41] Fig. 3 ）



datacenter tax 内部： protobuf 、压缩、 RPC 是三大项（[41] Fig. 5 ）

52%

端到端时间在远程工作（ 22% ） + 存储 IO （ 30% ）

存储 / 分析负载天然是 RPC 密集的 [41]

32–40%

CPU 周期是 datacenter tax

核心计算只有 18–36% ； 其余是 system tax [41]

32–42%

system tax： OS 、文件系统、标准库等

OS 18–28% ， 标准库最高 53% [41]

20–25%

tax 内部 protobuf 占比；压缩占 14–31%

RPC 占 tax 23–37% （数据库平台） [41]

为什么重要：第 8 页的表是 fleet 平均或在线微服务口径；这一页是存储 / 分析侧的画像。两边合起来才是“通用负载”的全貌。
解释了“序列化 ≠ RPC”： protobuf 和压缩的大头在存储平台的读写路径（行、日志、文件格式库），大块压缩数据在关键路径上。

含义：序列化和压缩的加速方案必须能被存储引擎 / 文件格式库调用，只挂在网络 RPC 路径上会漏掉 65–84% 的序列化周期。

数据来源与口径： [41] Google ISCA'23 ， Spanner / BigTable / BigQuery 三个生产平台的实流量刻画；百分比为各平台 CPU 周期归类（不是 fleet 平均）。

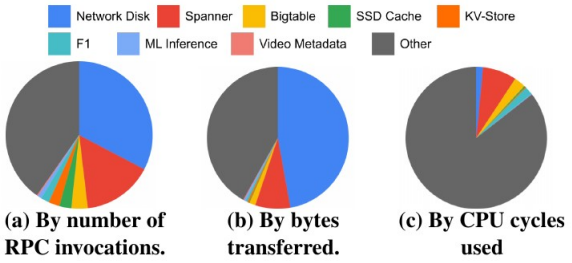
RPC 占比必须分三个口径讲：网络流量 95%+ 、 CPU 周期 7.1% 、延迟平均 2% （但尾部 38–96% ）

口径一：网络流量 —— RPC 是 DC 通信的主体

95%+

Google 生产 DC 的应用流量由 RPC 产生

其中约 75% 与存储系统交互 [17]



Google 头部 RPC 服务：按调用数 / 字节 / CPU 周期（[02] Fig. 8 ）

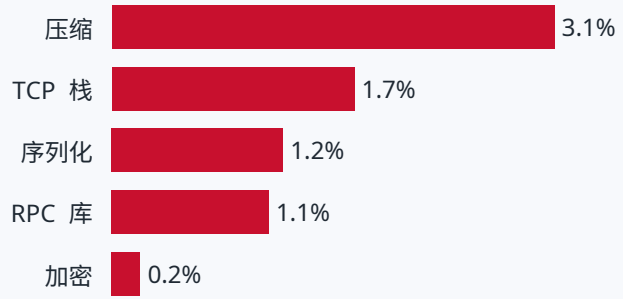
- 存储是主体中的主体： Network Disk 一个服务占 35% 调用但 <2% 周期——“量大、单个便宜”。

口径二： CPU 周期 —— RPC 税 ≈ 7.1%

7.1%

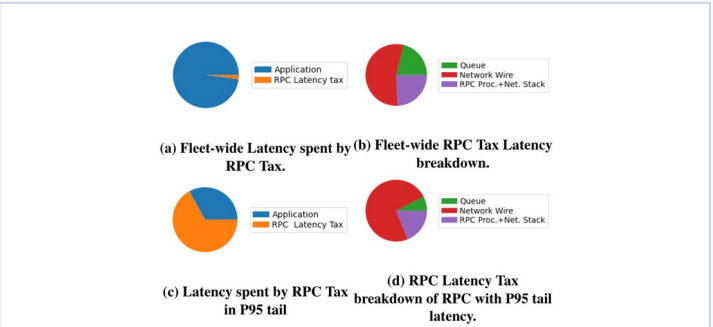
Google fleet 全部 CPU 周期中的 RPC 税

分解（占 fleet 周期 %） [02] ：



- 若把 RPC 路径上的 memcpy 、分配也算进去，“ RPC 全路径”约 10–15% （本报告推算）。
- Meta 头部服务是分布的另一端： RPC 收发 I/O + 序列化 + 压缩合计 Cache2 ≈59% 、 Web ≈59% 、 Feed1 ≈28% [20] 。

口径三：延迟 —— 平均只占 2% ，尾部由税主导



上：fleet 平均 RPC 延迟税 2% ；下： P95 尾部里税占比大幅上升（[02] Fig. 10 ）

平均 2.0% ： 网络 1.1% 、栈 0.49% 、排队 0.43% ；中位方法 8.6% 。

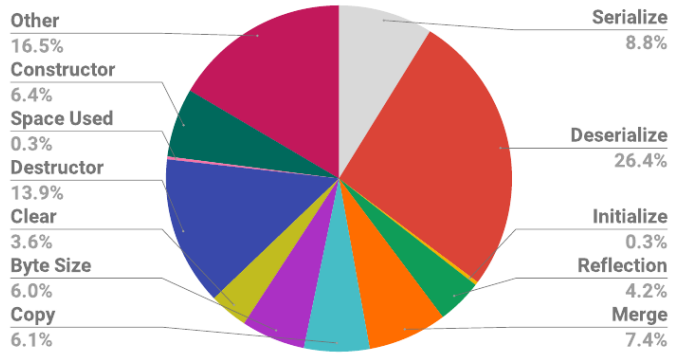
尾部 38% / 96% ： 开销最高的 10% 方法，中位税 38% 、 P90 96% ；排队占税的 21% 。

原因： 生产 RPC 是 ms 级的（ 90% 服务中位延迟 ⩾ 10.7 ms ），μs 级的栈开销被应用处理淹没。

Google 是 fleet 平均且服务偏“重”， Meta 是 I/O 密集的头部服务——两者不是矛盾而是同一分布的两端；引用时必须说明是哪个口径。

数据来源与口径： [17] Google SIGCOMM'22 （ 2021 年生产流量）； [02] Google SOSP'23 ， 1 万个方法、 7,220 亿条 Dapper trace ， CPU 归因用 GWP ； [20] Meta ASPLOS'20 ， 7 个微服务的叶子函数归因。

一个必要的修正：序列化 ≠ RPC——protobuf 9.6% 里只有约 1% 在 RPC 路径上；引用同一个数时要回到原文口径



Google fleet 的 C++ protobuf 周期按操作分解：反序列化 26% 、序列化 9% （[25] Fig. 2）

protobuf 占 9.6% fleet 周期，其中反序列化 2.2% 、序列化 1.25% 可加速，其余是“胶水代码” [25] 。

但只有 16% 的反序列化、 35% 的序列化来自 RPC 栈，其余来自 Bigtable / Spanner 的行、日志、进程内传递 [25] 。

三角验证： RPC 路径序列化 ≈ 16.3%×2.2% + 35.2%×1.25% ≈ 0.8% ，与 Cloud-Scale 独立测得的 1.2% [02] 同一量级。

因此本报告把两项分开列：“序列化税” ≈ 9.6% 与 “ RPC 税” ≈ 7.1%（含序列化 1.2% ），两者只有约 1% 重叠。

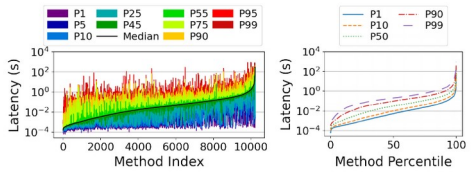
同一个数的不同说法（本调研反复遇到）

原文数字	被引用成	提醒
Kanev 2015： protobuf ≈3–4% （读图值）[01]	ProtoAcc 引为 5%； Zerializer / Optimus Prime 引为 “ RPC+protobuf 12%”； mRPC 引为 “ gRPC 库 10%”	引用时回到原文读图
Accelerometer： 序列化按服务 4–13% [20]	zBuffer / Cornflakes 引为 “序列化 6.7%”	是 7 个微服务的某种平均，不是 fleet
DCPerf： Meta 各服务 tax 18–82% [05]	常被当作 fleet 平均	是各服务的范围
DeathStarBench： 网络处理 36% [06] ； Cerebros： RPC 层 40–90% [13]	常与 Google 7.1% 并列比较	轻业务逻辑 + 已优化传输的设定，比 fleet 高一个数量级；对应 “热点服务” 而非 “ fleet 平均” 目标
阿里 2021 公开 trace [04]	大量论文据此做调用图分析	99.48% 的拓扑不一致问题 [40] ，引用其调用图结论需注明
Google RPC 税 7.1% [02]	被当作 “ RPC 不重要” 的证据	是 library 形态、 fleet 平均、 ms 级服务的下界； sidecar 用户和 KV 前端完全不同（第二部分）

读数规矩：先问 “谁测的、 fleet 平均还是热点服务、 RPC 路径还是全口径、中位还是 P99” ，再比较。

数据来源与口径： [25] Google MICRO'21 fleet GWP 采样； [02] Google SOSP'23 ； [20] Meta ASPLOS'20 ； [05] Meta ISCA'25 ； [06] DeathStarBench ASPLOS'19 （学术 benchmark ）； [13] Cerebros MICRO'21 （模拟）； [40] ICPE'24 对阿里 trace 的一致性检查。

RPC 长什么样（上）：以毫秒级为主、中位消息 <1.5 KB 但尾部几百 KB、少数方法承担大部分调用



按方法的 RPC 延迟分布（[02] Fig. 2）

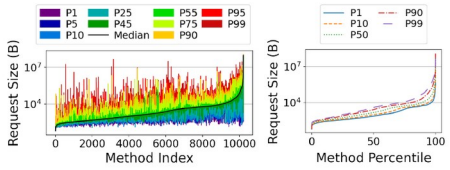
时间尺度

数据

90% 服务的中位延迟 ⩾ 10.7 ms； 99.5% 的方法 P99 ≥1 ms；最慢的 1,000 个方法只占 1.1% 调用但占 89% 的总 RPC 时间。

对硬件设计的含义

学术界“µs 级 RPC”的假设只对 KV / 缓存成立；在 fleet 里，省 CPU 比省 µs 更重要。



按方法的请求大小分布（[02] Fig. 6）

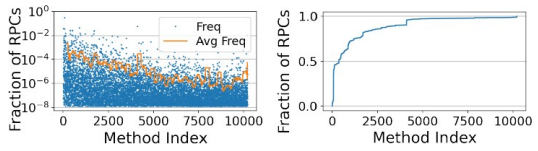
大小

数据

最小 64 B；一半方法的中位请求 <1.5 KB、响应 <315 B；P90 11.8 / 10 KB； P99 196 / 563 KB。 Homa 统计的 Google / Facebook 消息 >85% 小于 1,000 B [08]。

对硬件设计的含义

中位小到单 MTU、单 cache line 级；尾部重到几百 KB—— 卸载方案必须两头都处理，只处理单 MTU 会漏掉重尾。



按方法的调用频率与累计分布（[02] Fig. 3）

流行度

数据

top-10 方法占 58% 调用、 top-100 占 91%； Network Disk 的 Write 一个方法占 28%；该服务占 35% 调用但 <2% 周期。

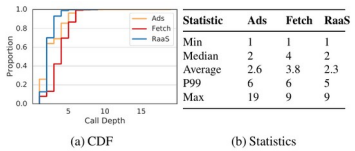
对硬件设计的含义

少数方法的专用优化就能覆盖大部分流量；存储 RPC“量大、单个便宜”，是 NIC 侧卸载最容易切入的形状。

一句话： fleet 里的 RPC 是“毫秒级、小消息为主但重尾、调用极度集中”——与 µs 级 KV benchmark 的画像相反，硬件目标应先对准 CPU 周期而非往返延迟。

数据来源与口径： [02] Google SOSP'23： 1 万个 RPC 方法、 7,220 亿条 Dapper 采样 trace，图中每个点是一个方法，颜色为该方法的分位数； [08] Homa SIGCOMM'18 引用的 Google / Facebook 消息日志。

RPC 长什么样（下）：调用树宽而不深、叶子几乎全是数据库、 CPU 成本与大小和延迟都不相关

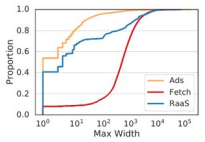


(a) CDF

Statistic	Ads	Fetch	RaaS
Min	1	1	1
Median	2	4	2
Average	2.6	3.8	2.3
P99	6	6	5
Max	19	9	9

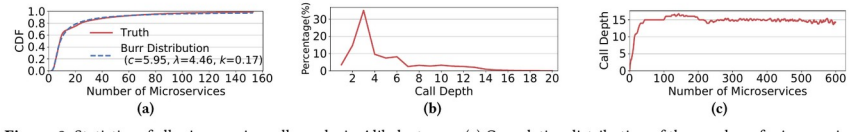
(b) Statistics

Meta： 调用深度中位 2–4 （[03] Fig. 11 ）

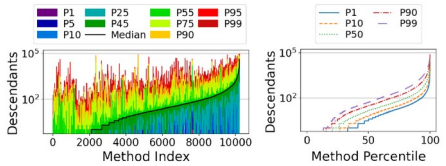


Statistic	Ads	Fetch	RaaS
Min	1	1	1
Median	1	472	3
Average	52	916	507
P99	774	7,395	7,885
Max	166,171	46,746	72,196

Meta： 最大宽度中位 472 、 P99 7,395 （[03] Fig. 12 ）



阿里 2 万服务：调用图规模 / 深度分布 / 深度 vs 服务数（[04] Fig. 3 ）



Google： 按方法的后代数分布，重尾（[02] Fig. 4 ）

调用树 · 数据

Google： 半数方法中位后代 ⩽ 13 、 P99 >1,155 ； 深度 P99 <10 [02] 。 Meta： Fetch 中位深 4 / 宽 472 ， P99 6 / 7,400 [03] 。阿里：常见深度 3 ， >10% 的图超过 40 个服务 [04] 。

含义

深度小 → 串行 RTT 累积有限，往返延迟不是主矛盾；宽度大 → 扇出 / incast 和叶子（存储）负载是主体。

叶子 · 数据

Meta： 叶子和单中继的服务执行占 53–73% ，叶子全是数据库 [03] 。阿里：叶子 61% 是有状态服务；大图里一半是 Memcached [04][39] 。

含义

RPC 服务端的主体是存储 / 缓存前端——最同质、物理上最集中（独立机群），是专用硬件最容易切入的位置。

CPU 成本 · 增速 · 错误

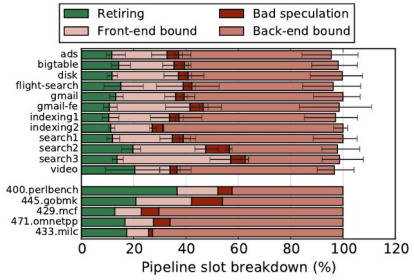
每方法 CPU 成本重尾： P99 比中位高 1–2 个数量级； RPC 大小、延迟都与 CPU 成本不相关。 RPS/CPU 每年 +30% 。 1.9% 的 RPC 出错， “Cancelled” 占 45% 错误、 55% 浪费周期（对冲请求） [02] 。

含义

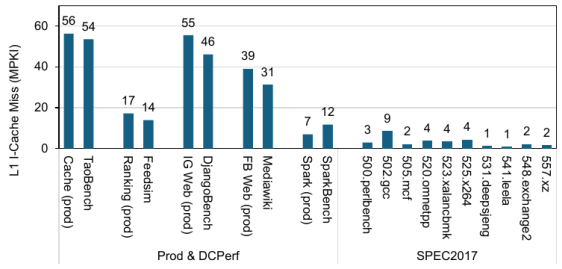
不能按大小预测成本 → 负载均衡只能动态 / 启发式； RPC 增长快于算力；尾延迟优化手段（对冲）本身有 CPU 代价。

数据来源与口径： [02] Google SOSP'23 ； [03] Meta ATC'23 （ Ads / Fetch / RaaS 三类 trace ， 18,500 服务 / 1,200 万实例）； [04] 阿里 SoCC'21 （约 2 万服务，注意其公开 trace 的拓扑一致性问题 [40] ）； [39] 阿里 2022 trace 。

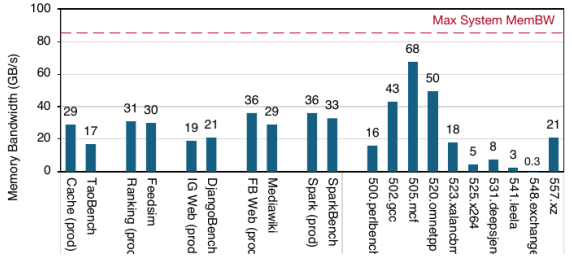
微架构画像： CPU 的钱花在前端取指、内核和上下文切换上，不在内存带宽



Google 12 个应用 vs SPEC 的 Top-Down 分解：前端 stall 明显更高（[01] Fig. 6 ）



Meta 生产负载 L1-I MPKI 7–56， SPEC 只有 1–9 （[05] Fig. 8 ）



Meta 生产内存带宽 19–36 GB/s，约为系统上限的 30% （[05] Fig. 7 ）

① 前端是第一瓶颈

Front-end bound 占 15–30% 流水线 slot （SPEC 的 2–3 倍）， 5–10% 周期完全取不到指令； i-cache 工作集每年 +27% [01]。 DCPerf： L1-I 只有 1/4 的 ARM SKU 在 Web 负载上 Perf/W 低 85%，被淘汰 [05]。

② 先进预取 / 分支预测对微服务几乎无效

Pythia 数据预取、 I-SPY 指令预取对单体 +16–19%，对微服务 ≈ 0 [36]。请求处理函数工作集小（0.5 MB， L1 命中 >95%），问题在切换和服务间足迹，不在顺序流。

③ 内存带宽严重过配，延迟才重要

带宽利用率中位 10%、 95 分位 31% [01]； 生产 19–36 GB/s ≈ 系统 30% [05]； 微服务主动不用满以免延迟指数上升。但少数热页贡献大部分带宽 → “带宽分层”内存 [40]。

④ 内核占比高

Google ≈1/5 周期 [01]； Meta Cache 类 ~30% 时间在内核 [05]； 内核 IPC 低且跨代不改善（厂商用 SPEC 调优） [20]； 384 核上 kernel 6.4 vs 6.9 差 54%—— 核数暴露锁争用 [05]。

⑤ IPC 普遍低， SMT 有效

生产 IPC 1.0–2.6 [05]； 每类叶子函数用不到峰值一半 [20]。

⑥ chiplet / NUCA 是新变量

跨 LLC 域的核间传输延迟是域内的 2.07×，迫使分配器分片 [35]； 跨核负载均衡让内核网络栈每阶段慢 2–3× [32]。

⑦ 对 CPI 敏感、对 LLC miss 不敏感

RPC 栈重的 KV 服务尾延迟随 CPI 波动；分支预测失败率和 LLC miss 率与 RPC 延迟不相关 [02]。

⑧ 主频杠杆与小核

Uber： 同微架构主频 +9% → P50 −15% 、尾延迟 − 1.5× [07]； ThunderX 顺序小核不可用于关键路径 [06]。但 μManycore 证明小核问题在软件调度 / 切换，解决后 iso-power 尾延迟 − 10.4× [36]。

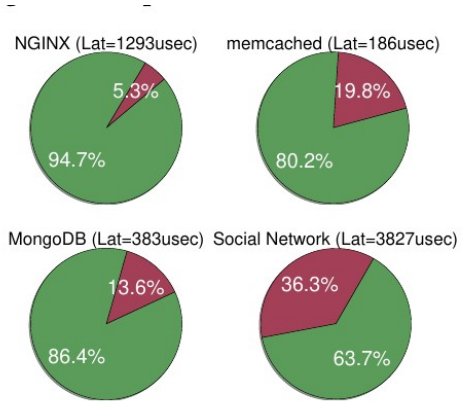
对芯片的含义：更大 / 更聪明的取指路径、更便宜的切换与唤醒、可划分的一致性域，比堆内存通道和更强的顺序预取器更贴近微服务的瓶颈。

数据来源与口径： [01] Google ISCA'15 （Top-Down， Ivy Bridge）； [05] Meta ISCA'25 DCPerf （生产 SKU2， 2024）； [36] μManycore ISCA'23 与 SoftSKU ISCA'19； [20] Meta ASPLOS'20； [35] Google ASPLOS'24； [32] Homa/Linux ATC'21； [02] Google SOSP'23； [07] Uber ATC'22； [06] DeathStarBench ASPLOS'19。

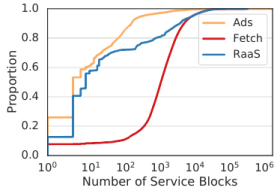
RPC 负载是怎么产生的：微服务架构与部署形态

每请求 RPC 数（拆分粒度与调用图）→ 每 RPC 成本 × 部署形态 A （RPC 栈跑在哪）→ 放置与拓扑（形态 C）→ VM / 容器 / serverless （形态 B）→ 三角验证与数据空白 → 负载洞察小结

每请求 RPC 数：拆分即税——同一功能，单体的网络处理只占几个百分点，微服务版占 36%；生产系统一次请求中位数百次调用



单体 5–20% vs 微服务版 Social Network 36.3% （[06] Fig. 3）



(a) CDF

Statistic	Ads	Fetch	RaaS
Min	1	1	1
Median	2	498	4
Average	83	1,052	628
P99	1,233	8,911	10,667
Max	253,877	81,859	113,790

(b) Statistics

Meta：一次 Fetch 请求中位 498 个服务块、P99 8,911 （[03] Fig. 10）

公司	服务规模	每请求 RPC 数	调用图形状	来源
Meta	18,500 服务 / 1,200 万实例（去噪后 7,400 / 1,120 万）；复制中位 8、最大 55.7 万	Fetch 中位 498 次调用（13 个唯一服务）、P99 宽 7,400； Ads 中位 2	浅（深 4–6）而宽（472–7,400）；叶子全是 DB	[03]
阿里 2021	约 2 万服务	常见深度 3； >10% 的图 >40 个服务；大图一半是 Memcached	树状、入度 1； 5% 超级服务处理 95% 调用	[04]
阿里 2022	64,760 服务 / 187 万实例；中位 4 实例	—	密度比 Meta 稀疏 10× 但局部度分布相似； top 1% 服务 = 87% 调用	[39]
Google	1 万 + RPC 方法	半数方法中位后代 ⩽ 13； P99 >1,155	宽 > 深；深度 P99 <10	[02]
Uber	4,000 服务 / 40,000 端点	单 trace 数千个 RPC	深且异步；关键路径只是子集（一个端点 68% 的 P50 延迟来自单个下游）	[07]

Dagger 在同一 benchmark 上：通信占各微服务延迟平均 40%，轻逻辑层达 80%； RPC 处理占各微服务执行时间 5–75%，高负载下占端到端尾延迟 >30% [12][06]。

一个修正：对延迟而言只有关键路径上的 RPC 算数（CRISP：一个端点 68% 的 P50 延迟来自单个下游 [07]）；对 CPU 而言全部 RPC 都算数。

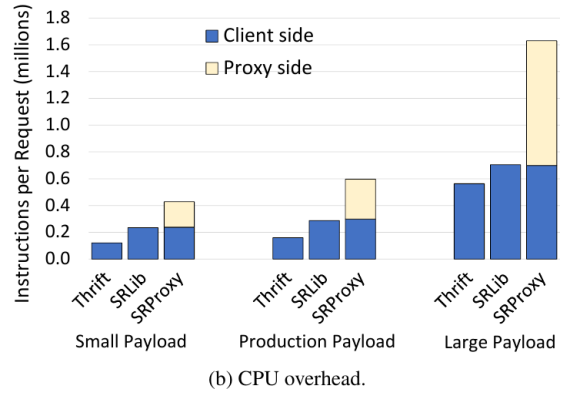
减少 RPC 数的软件手段：阿里 7.8% 的依赖是环形、2.7% 强耦合——合并接口可把远程调用变本地 [04]； Google Service Weaver 让运行时决定同进程还是跨进程 [40]； Meta FaaS 同进程跑多个函数 [22]； CXL 共享内存让同机架调用退化为内存操作 [14][29]。

三个跨公司共性：(a) 宽不深——扇出到分片存储是 RPC 数量的主要放大器；(b) 热点极度集中——少数超级服务 / 方法承担 87–95% 的调用；(c) 叶子是存储—— RPC 服务端的主体是 DB / 缓存前端。

数据来源与口径：[06] DeathStarBench ASPLOS'19（学术 benchmark，单机）；[03] Meta ATC'23（Ads / Fetch / RaaS 三类生产 trace）；[04][39] 阿里 SoCC'21 / arXiv'25；[02] Google SOSP'23；[07] Uber ATC'22 CRISP；[12] Dagger ASPLOS'21。

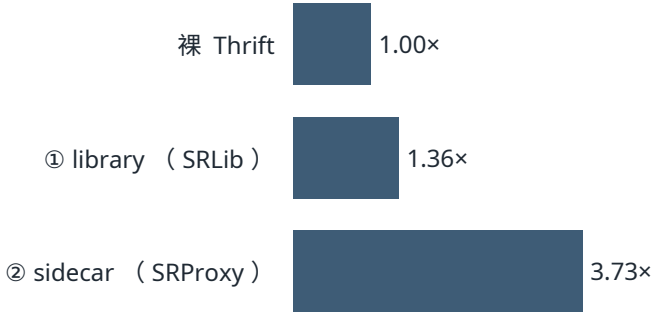
RPC 栈跑在哪（部署形态 A）决定每 RPC 成本：以进程内 library 为基线， sidecar 让 CPU +273%，硬件卸载让传输层 CPU ≈ 0

形态	成本落在哪块硅	实测系数（基线 = 进程内 library 直连、裸传输）	来源
① 进程内 library （Stubby / SRLib / gRPC-in-app）	应用核 + 内核网络栈	生产聚合 +36% CPU （最坏微基准 +80%） vs 裸 Thrift；延迟 ≈ 裸 Thrift	[21]
② sidecar / 本地代理（Envoy / Istio，SRProxy）	额外的代理核 + 两次额外内核穿越 + 编解码 ×3	CPU +273%、延迟 +107% （Meta）； gRPC+Envoy 中位延迟 63→203 µs、 P99 +180%、带宽 −44%；应用延迟 +27–269%、 vCPU +42–163%；集群 10% CPU / 内存，极端 3× 于应用（阿里云生产）；吞吐 −56%、延迟 2.27× （XLB）	[21][18][23][24][39]
③ 每节点 / 集中式代理（Ambient、Canal 网关、 SRRemoteProxy）	共享代理核（本节点或云网关）	Ambient 比 Istio 代理数 −43%； Canal 比 Istio CPU −12–19×、比 Ambient −4.6–7.2×；但多一跳远程	[24]
④ RPC 作为节点级系统服务（Snap / Pony Express、 mRPC）	独占 / 共享的服务核 + 共享内存跳	Snap： Gbps/ 核 比内核栈 3×，但比独占栈低 3.5–7×， 100 Gbps 需 7–14 核； mRPC 比 gRPC+Envoy 快 6.1× 但比 eRPC 慢 2× （+5 µs IPC）	[38][18][32]
⑤ 内核旁路 library （eRPC、Junction）	应用核（独占轮询或 Junction 式共享）	2.3 µs、 10 Mrps/ 核（100 Gbps 内核栈需 18 核， eRPC 约 1–2 核）； Junction 密度 19–62×	[31][32][40]
⑥ 内核内 L7 （XLB eBPF）	应用所在核，无跨进程	比 Istio / Cilium 吞吐 +1.5×、延迟 − 60%；生产延迟 − 41%、密度 +30%	[39]
⑦ NIC / 加速器卸载（Dagger、Cerebros、 RpcNIC、 Falcon）	NIC / IPU / 专用单元	传输层 CPU ≈0 （Falcon）； RPC 层 CPU −74% （RpcNIC， PCIe）到 − 37–64× （Cerebros，片上模拟）	[30][28][13]



Meta 生产：每请求指令数， Thrift / SRLib / SRProxy （客户端 + 代理侧）（[21] Fig. 10b）

每 RPC CPU 成本倍数（Meta 生产，裸 Thrift = 1）

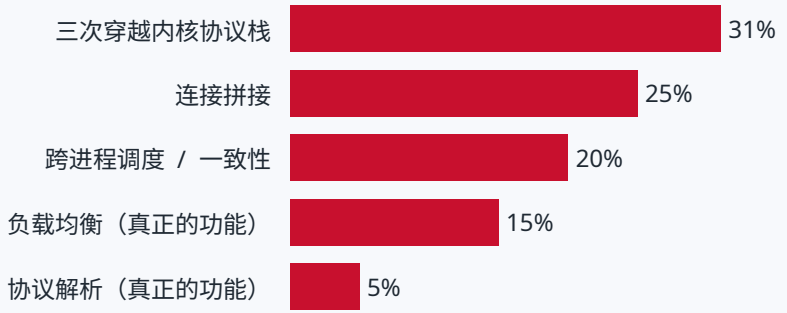


演化方向的共同点： sidecar （每 pod）→ 每节点代理 → 集中网关 → 内核内 / 硬件——每一步都把 L7 处理集中到更少、更专业的位置，与硬件卸载同向。

数据来源与口径： [21] Meta OSDI'23 ServiceRouter （生产聚合）； [18] mRPC NSDI'23； [23] MeshInsight SoCC'23 （benchmark）； [24] 阿里云 SIGCOMM'24 Canal （生产）； [39] XLB arXiv'26； [38] Snap SOSP'19； [32] Homa/Linux ATC'21； [31] eRPC NSDI'19； [40] Junction NSDI'24； [30] Falcon SIGCOMM'25； [28] RpcNIC HPCA'25； [13] Cerebros MICRO'21 （模拟）。

sidecar 为什么贵：三份独立分解一致—— L7 语义本身不贵，贵的是多出来的进程边界、协议栈穿越、重复编解码和拷贝

XLB 对 sidecar 开销的分解 [39]： LB 功能只占 20%



ServiceRouter [21]： 代理开销 74% 在用户态（序列化、加密、身份），零拷贝也救不了。

MeshInsight [23]： HTTP / gRPC 模式下协议解析占 63–77%、TCP 模式下 IPC + socket 写占主导；零拷贝 socket 写对 <4 KB 消息几乎无效。

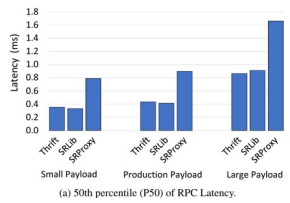
各家怎么选：形态由“信任边界”决定，不是性能

Google / Meta 内部默认 ① library。 Meta 99% 的 RPC 走 SRLib，1% 走代理（只用于 Erlang 等无法链接的运行时）；全量改代理要多 O(100,000) 台机器 [21][22]。Google 的 Stubby 是 library，其下是 ④ Snap 用户态服务，正演化为 ⑦ Falcon 硬件 [38][30]。

hyperscaler 的 7.1% RPC 税是 library 形态下的数字，是下界。

公有云租户以 ② sidecar 为主（CNCF：90% 组织在用或评估 mesh [23]），因为不能改应用、多语言、需要强制策略；阿里云运营 Istio 四年后转向 ③ 集中网关 [24]；业界 2024–2026 的方向是 ③ Ambient / Cilium 与 ⑥ eBPF [24][39]。80–95% 的 mesh 用户需要 L7 功能 [24]，所以纯 L3/L4 的 eBPF 不够。

library 形态不能服务外部客户（无隔离、要改应用） [39]—— 形态选择本质是信任边界决定的。



Meta： P50 延迟 Thrift ≈ SRLib， SRProxy +107% （[21] Fig. 10a）

L7 处理位置的演化：集中到更少、更专业的位置

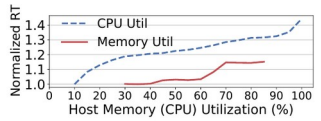
sidecar（每 pod）	每节点代理	集中网关	内核内 L7	NIC / DPU 卸载
Istio：集群 10% CPU / 内存 [24]	Ambient：代理数 − 43% [24]	Canal：CPU 比 Istio −12–19× [24]	XLB eBPF：生产延迟 − 41% [39]	L7 元数据随包交付（待验证）

综合：贵的不是 L7 逻辑，而是为了放置 L7 逻辑多出来的进程边界、协议栈穿越、重复编解码和拷贝——这也是为什么零拷贝对小消息几乎无效，而“减少跳数”有效。

数据来源与口径：[39] XLB arXiv'26（sidecar 开销分解）；[21] Meta OSDI'23（生产）；[23] MeshInsight SoCC'23（benchmark + CNCF 调查）；[24] 阿里云 SIGCOMM'24（生产运营数据）；[22] Meta CACM'25；[38] Snap；[30] Falcon。

放置、拓扑与隔离环境（形态 B / C ）： 混部干扰、 NUMA 、 CXL 机架和 serverless 都会改变每 RPC 成本的系数

形态 C： 放置与拓扑



阿里：主机 CPU 利用率 10→30% ，在线服务 RT 劣化 20% （[04] Fig. 14 ）

有状态服务独立部署： 阿里 DB / Memcached 独立集群 [04]； Meta 存储服务器是单独 SKU [05] → RPC 的叶子物理上集中在存储机群，加速器最容易切入。

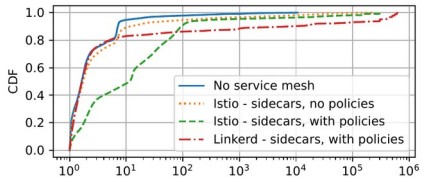
混部干扰： 阿里在线容器 CPU <10% ，但主机利用率 10→30% 就让 RT 劣化 20% ，内存利用率 <60% 无影响 [04]； 一个节点平均托管 15 个服务的实例 [39] → 核 / LLC / 带宽的硬件隔离能力决定混部收益。

区域内 vs 跨区域： Meta 区域内跨 DC 无性能惩罚（ fabric 无超订） [22]； Google 跨集群 RPC 的网络延迟就是光速延迟（不是拥塞），可优化的是数据局部性 [02] 。

CXL 机架： HydraRPC 真硬件 1.47 µs [14] 、 RPCool 1.5 µs （ RDMA 回退 17 µs ） [29] → 同 CXL 域内 RPC 成本降一个数量级；但 RPCool 在 DeathStarBench 上与 Thrift 持平—— DB 更新主导时通信优化无效。

NUMA / NIC 位置： NIC 远端 NUMA 节点吞吐 / 核 − 20% [33]； Snap 把引擎绑到 NIC 所在节点——零拷贝下 NIC 局部性 > 应用局部性 [38] 。

形态 B： VM / 容器 / serverless



serverless + mesh 策略：每次调用慢 7×（p50 ）到 254× （p99 ）（[43] Fig. 1 ）

容器 + 裸金属是主流： 阿里 K8s 裸金属 [04] 、 Meta Twine 容器 + 本地 SSD [22] 。

Meta 的 FaaS 常驻 >50 万台、无冷启动、同进程多函数—— hyperscaler 独有的效率手段 [22]； 公有云每函数一 VM 的隔离代价高（ Lambda 上用 S3 传状态延迟显著更高） [06] 。

serverless × sidecar 的乘积： Azure Functions 生产 trace 在 Knative 上加 Istio / Linkerd 常见策略（ mTLS 、授权、认证、限流）后 p50 慢 7× 、 p95 38.3× 、 p99 254× ， 90% 调用受影响，每节点多耗 2.5–6.5 核；不加策略时中位只 +6% 但 p99 仍 55×；冷启动 1.2 s → 4 s ； 最贵三项 = mTLS 、授权、请求认证 [43] 。比长驻微服务的 sidecar 系数（ 2.7–3.7× ）高一个数量级：函数短、策略重、沙箱与策略高频增删。

IOMMU 税： 虚拟化必开的 IOMMU 让网络栈吞吐 / 核 − 26% （每页 map / unmap ，接收端内存管理 30% 周期） [33] 。

形态系数从 library 1.36× → sidecar 2.7–3.7× → serverless + 策略 7–38×： 每往外一层隔离， RPC 成本就高一个量级。硬件能缩小差距的是廉价的隔离 / 切换与快速冷启动，而不是更快的网线。

数据来源与口径： [04] 阿里 SoCC'21 （生产）； [39] 阿里 2022 trace ； [05][22] Meta ； [02] Google ； [14] HydraRPC ATC'24 （真 CXL FPGA ）； [29] RPCool arXiv'24 （ NUMA 模拟）； [33] SIGCOMM'21 内核栈刻画； [38] Snap ； [43] ETH SoCC'25 （ Azure Functions 生产 trace 在 Knative 上重放）； [06] DeathStarBench 。

三角验证：能从部署形态占比推出负载全景吗？——能部分验证、不能完整推导；三个数据空白正是芯片厂商可以和客户一起测的

① Meta 的形态加权

99% × 1.36 （library） + 1% × 3.7 （代理） ≈ 1.38× 裸 RPC。即 Meta fleet 的 RPC 成本几乎完全由 library 形态决定，代理的 1% 流量只贡献约 3% 的 RPC 成本 [21][22]。

② 公有云 sidecar 租户的外推

library 形态下 RPC 税 ≈ 7% （Google [02]） × sidecar 系数 2.7–3.7 → RPC 税 ≈ 19–26%，增量 ≈ 12–19% 的 CPU。阿里云生产实测 sidecar 占集群 10% CPU / 10% 内存（极端 3× 于应用） [24]，MeshInsight 在 benchmark 上 +42–163% vCPU [23]—— 量级吻合。

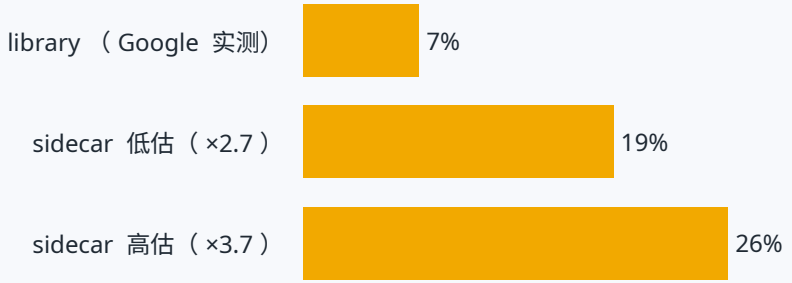
③ 序列化的两份独立测量

ProtoAcc 由 fleet 数据推算 RPC 路径序列化 ≈ 0.8% [25]； Cloud-Scale 直接测得 1.2% [02]——吻合（差异来自年份和归类）。

④ 调用数 × 单价的粗算

Meta Fetch 中位 498 次调用 / 请求 × 每次 RPC 若省 5 µs CPU = 2.5 ms / 请求 [03]。对一个 P95 SLO 500 ms 的排序服务是 0.5%，对 ms 级 KV 是 100%+ → 同一个“每 RPC 成本”的改进，收益完全取决于服务类型。

RPC 税按部署形态外推 & 数据空白



RPC 税占 CPU 周期（%）：library 为 Google fleet 实测，sidecar 两档为按 Meta / mRPC 系数外推，与阿里云生产 10% 集群 CPU 同量级

数据空白（可与客户联合测量）：

- (a) 各公司 fleet 中 library / sidecar / serverless 形态的流量与 CPU 占比
- (b) 每 RPC 成本按形态的 fleet 加权分解
- (c) Java / 托管语言环境（阿里、大数据）的 RPC / 序列化周期
- (d) sidecar 卸载到 DPU 后的真实 CPU 节省

公开数据缺“各形态在 fleet 中的流量占比 × 单位成本”这一乘法的完整输入——这是三角验证无法闭合的原因，也是芯片厂商最有价值的第一手测量。

数据来源与口径：外推为本报告计算（REPORT.md 2.5 节），输入数据分别来自 [21][22] Meta、[02] Google、[24] 阿里云、[23] MeshInsight、[25] ProtoAcc、[03] Meta 拓扑； sidecar 系数 2.7–3.7 取自 [21][18]。

负载洞察小结：从生产数据得到的八条结论

1 没有杀手应用，只有“税”值得硬件专用化

top-1 程序占 10%、top-50 占 60% 且逐年下降（Google）；应用内部同样平坦。跨应用共享的底层例程是唯一可寻址的大块。

2 税的量级排序（fleet 平均）

内核 ~20%（不算可加速税）> 序列化全口径 9.6% > 内存分配 4.3% ≈ memcpy 4–5% > 压缩 2.9% > TCP 栈 1.7% > RPC 库 1.1%。若只看 RPC 路径：压缩 3.1% > TCP 1.7% > 序列化 1.2% > RPC 库 1.1%。

3 RPC 占比有三个口径

网络流量 95%+（75% 与存储交互）；CPU 周期 7.1%（library 形态、fleet 平均，是下界；Meta 头部服务 28–59%）；延迟平均 2% 但开销最高的 10% 方法里税占 38–96%。

4 序列化 ≠ RPC；存储平台是税的另一半

65–84% 的序列化发生在存储 / IPC 路径；Spanner / BigTable / BigQuery 的周期里核心计算只有 18–36%，protobuf 与压缩是 tax 的大项。

5 RPC 的形状

ms 级为主；中位 <1.5 KB 但 P99 几百 KB；top-100 方法占 91% 调用；调用树宽不深、叶子全是数据库；CPU 成本与大小、延迟不相关。

6 微架构：钱花在前端和内核

前端 stall 15–30%、生产 L1-I MPKI 7–56（SPEC 1–9）；先进预取对微服务无效；带宽利用率只有 10–31%；内核 20–30%；Cache 类切换最多 18%。

7 部署形态系数决定同一 RPC 的成本

library 1.36× → sidecar 2.7–3.7× → serverless + 策略 7–38×。形态由信任边界决定；演化方向是把 L7 处理集中到更少、更专业的位置，与硬件卸载同向。

8 三个数据空白

形态在 fleet 中的占比、托管语言（Java）环境的 RPC / 序列化税、网络：存储序列化的比例——公开文献都没有，芯片厂商应与客户联合测量。

对硬件目标的含义：先对准 CPU 周期（序列化 + 压缩 + 拷贝合计 15–20%）与前端 / 切换效率，其次才是往返 µs；最同质的落点是存储 / 缓存前端，最大的增量市场是 sidecar / serverless 用户。

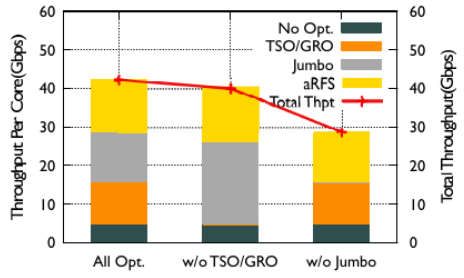
数据来源与口径：各条结论的原始数据与口径见第 6–20 页对应页脚；数字均为论文原文或本报告的显式推算。

三

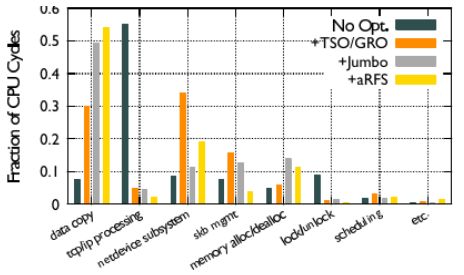
学界怎么削： RPC 通信栈与序列化的研究方向

通信栈的成本机理 → 五条削减路线（传输协议 / 内核旁路 / 服务器端调度 / 硬件卸载 / CXL ）→ 硬件卸载谱系的四条规律 → 序列化的成本结构 → 序列化加速器的放置之争与压缩

通信栈的成本机理：主机侧网络成本 = 拷贝 + 跨核分发 + cache 争用 + 地址翻译，而不是协议计算



单长流：全部优化开启也只有约 42 Gbps/ 核（[33] Fig. 3a ）



(d) Receiver CPU breakdown

接收端 CPU 分解：数据拷贝 >50% （[33] Fig. 3d ）

单核已跑不满线速。 100 Gbps 下 Linux 内核栈最优约 42 Gbps/ 核（jumbo + TSO/GRO + aRFS 全开）；接收端内核→用户的数据拷贝占单长流 >50% 周期； 400 Gbps 下 payload 拷贝 >60% CPU 、仅 memcpy 就需 ⩾ 15 核 [33] 。

协议逻辑不是主要成本。 Homa 做进 Linux 内核后拥塞被协议消除，但短消息往返软件开销约 9.5 µs （> 网络 5.6 µs ），分散在各组件、无单一热点；跨核负载均衡（ RSS → NAPI → SoftIRQ 两级哈希）使每阶段慢 2–3× ； 100 Gbps 双向 80% 需 ⩾ 18 核 [32] 。

LLC 是被忽视的 I/O 瓶颈。 DDIO 只用 2 个 LLC way ， BDP 增长超过 LLC 增长 → 单流 49% cache miss 、吞吐 / 核 − 24% ； 200 Gbps × 1 KB 包 = 每 41.8 ns 一次 I/O ，一次 miss 就破线速； CEIO 用“总信用 = LLC 容量”实现零 miss ，吞吐 +1.2–2.9× [33] 。

其他系数： IOMMU −26% ； NIC 远端 NUMA −20% ；长短流混跑同核 − 43% ；拥塞控制算法的选择对吞吐 / 核无影响——瓶颈在接收端 [33] 。

TCP 的六宗罪（ Ousterhout ）： 连接开销、字节流 HOL 、公平调度抬高尾延迟、发送端驱动 CC 保证高负载下有排队、不用网内优先级、按序投递限制负载均衡；且实现太复杂难以硬件化 [32] 。

四项成本分别对应四个硬件设计空间（第四部分发力点编号）

拷贝

DMA 引擎 / 零拷贝（ A2 、 B3 ）

跨核分发

NIC 按实时核负载动态分发（ B2 ）

cache 争用

LLC 信用 / 分区、 NIC 可见的占用反馈（ A5 ）

地址翻译

IOMMU / ATS 、大页 DMA 、更大 IOTLB （ A7 ）

对硬件的含义：主机侧网络成本 = 拷贝 + 跨核分发 + cache 争用 + 地址翻译，而非协议计算——所以“更快的 TCP 卸载引擎”不是答案，数据路径和缓存路径才是。

数据来源与口径： [33] SIGCOMM'21 Host Network Stack Overheads （ 100 Gbps 测试床， Linux 5.4 ）及 CEIO SIGCOMM'25 / FlexiNS arXiv'25 的 400 Gbps 数据； [32] Homa/Linux ATC'21 （ 40 节点集群， 25/100 Gbps ）。

通信方向的五条路线：传输协议重设计、用户态 / 内核旁路、服务器端调度与过载控制、硬件卸载、CXL 共享内存

① 传输协议重设计：消息语义、接收端驱动、网内优先级

Homa （SIGCOMM'18）：真实 RPC 消息 >85% <1 KB；盲发首个 RTTbytes + 8 级交换机优先级近似 SRPT + 接收端逐包授权，短消息 P99 比 TCP 低 100× [08]。Aequitas （Google 生产）：RPC 大小与业务优先级不相关，按 RPC 标 QoS + 发送端 AIMD 准入，fleet P99 −10%，无需新硬件 [17]。Falcon （Google IPU）：200 Gbps、120 Mops/s、拥塞下比 RoCE 好 8×、IP 只占 NIC 面积 5–6% [30]。

② 用户态与内核旁路：软件能做到多好

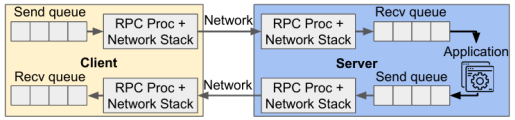
eRPC 是软件下界：2.3 µs、10 Mrps/ 核、有损以太网不靠 PFC，但独占轮询核、无隔离 [31]。Snap （Google 生产）：栈作为用户态服务，首要动机是迭代速度，代价是 100 Gbps 双向需 7–14 核 [38]。mRPC：序列化 + 策略搬进节点级服务，比 gRPC+Envoy 快 6.1× 但比 eRPC 慢 2× （+5 µs IPC） [18]。Junction：解决内核旁路的密度问题，每机多跑 19–62× 实例 [40]。

③ 服务器端调度、分发与过载控制

Google：排队占 RPC 延迟税 21%，>50 µs 的调度唤醒与尾延迟强相关 [02]。RPCValet：RSS 静态分流浪费 30–76% 吞吐，NI 侧“replenish 即空闲”推送分发距最优 3–16% [11]。nanoPU：NIC 直达寄存器文件 69 ns、硬件线程调度 <1 ns 决策，尾延迟 − 5× [10]。µManycore：1024 小核 + 硬件队列 / 切换，尾延迟 − 10.4× [36]。Breakwater：服务端信用准入 <20 ms 收敛 [16]。

⑤ CXL 共享内存 RPC （④ 硬件卸载见下页）

HydraRPC （真 CXL 硬件） 1.47 µs、大消息比 RDMA RPC 快 96–247×，主动绕开 CXL.cache 用非缓存共享 [14]；RPCool 1.5 µs （RDMA 回退 17 µs），只有 ⩾ 2 页的参数 seal 才比 memcpy 划算，DeathStarBench 上与 Thrift 持平 [29]。限制：CXL 域内（机架）、同 ABI、通知只能轮询。



Google 的 RPC 延迟分段模型：发送队列 → RPC 处理 + 网络栈 → 网线 → 接收队列 → 应用 → 回程（[02] Fig. 9）

表 3-8 通信方向：方案 × 削减项 × 成熟度

削减的成本项	软件方案	硬件方案	成熟度
网络排队 / 尾延迟（网线段）	Homa、Aequitas （生产）	Falcon （量产 IPU）、nanoPU 硬件 CC	生产
内核栈 CPU （TCP 1.7%、拷贝）	eRPC、Snap （生产）、Junction、Homa/Linux	Falcon、FlexiNS、CEIO （LLC 信用）	生产 / 原型
服务器排队与分发（接收队列段）	R2P2 JBSQ、Breakwater、Protego	RPCValet / NeBuLa / nanoPU 硬件分发与调度、µManycore	原型 / 模拟
RPC 库 + 分发（1.1%）	mRPC	Cerebros、Dagger、RpcNIC	原型
跨进程 / sidecar 跳数	mRPC、XLB （eBPF）、Canal 集中网关	DPU L7 （待验证）	部分生产
通信 + 拷贝 + 序列化（CXL 域内）	HydraRPC、RPCool	CXL 控制器功能	早期

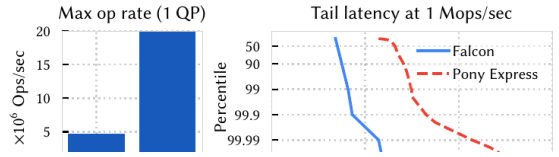
五条路线的共同模式：面向消息 + 无连接；接收端驱动的信用 / 授权（Homa 的 grant、Breakwater 的信用、R2P2 的 JBSQ、Aequitas 的准入分别控制链路、服务器 CPU、网络优先级）；把需要全局状态的功能（CC、pacer、分发）下沉到 NIC / 硬件——这一跳只能靠硬件消除 [32]。

Google 的取舍：Falcon 明确否定接收端驱动路线（假设网络无超订）和包喷洒（要求路径同构），保留面向连接但用内存带宽 + 二级缓存压平 >100K 连接的悬崖 [30]。

数据来源与口径：[02] Google SOSP'23；[08] Homa SIGCOMM'18；[17] Aequitas SIGCOMM'22 （Google 50 集群生产）；[30] Falcon SIGCOMM'25 （IPU E2100 实测）；[31] eRPC NSDI'19；[38] Snap SOSP'19；[18] mRPC；[40] Junction / R2P2 / Protego / NeBuLa；[11] RPCValet ASPLOS'19 （模拟）；[10] nanoPU OSDI'21 （FPGA）；[36] µManycore （模拟）；[16] Breakwater；[14] HydraRPC ATC'24；[29] RPCool arXiv'24。

硬件卸载谱系（FPGA NIC → 片上 NI → IPU）：接口语义比物理带宽更重要、小任务必须由 NIC 自主触发、 DPU 的 ARM 核不是加速器

方案	卸载内容	CPU– 设备接口	关键结果	主要限制
DeathStarBench FPGA [06]	TCP 栈（bump-in-the-wire）	PCIe	网络处理延迟 – 10–68×，端到端尾延迟 – 43%–2.2×	只卸传输，未报告 CPU 节省
Dagger （ASPLOS'21） [12]	整个 RPC 栈（接口 / 序列化 / 分发 / 传输）	UPI 内存互连（类 CXL）：CPU 只写共享 buffer	每核 12–16 Mrps、 2.1 µs；内存互连比 doorbell +14%； memcached 延迟比内核低 11.4×	MTU = 1 cache line；FPGA 回环
Cerebros （MICRO'21） [13]	Thrift RPC 层三模块（头 / 载荷 / 分发）	片上 NIC （NeBuLa）	RPC 层周期 – 37–64×，总周期 – 1.8–14×； i-cache MPKI –93%；面积比每核复制少 5×	全模拟； RPC 层 40–90% 是轻逻辑设定
nanoPU （OSDI'21） [10]	传输 + 核选择 + 调度 + 数据路径	寄存器文件	69 ns 线到线； 118 Mrps/ 核； +15% 核逻辑	只对 <1 µs cache 常驻请求
RpcNIC / RPCAcc （HPCA'25） [28]	RPC 层 + protobuf 序列化	PCIe SmartNIC （U280）	序列化 – 2.3× / –4.3×；主机 CPU –74%； RX 与片上持平、 TX 差 1.24–1.4×	主机需预序列化（22% 周期）； 250 MHz FPGA
FlexiNS （arXiv'25） [33]	可编程传输（BF3 ARM 只做包头）	PCIe， header-only TX	400 GbE 线速、单流接近 RDMA NIC	ARM 内存 / 链路带宽是硬约束
Falcon （SIGCOMM'25） [30]	可靠传输全部（CC / 重传 / 多路径 / 整形 / 加密 / 隔离）	PCIe （IPU E2100 ASIC 同 die IP），verbs / NVMe 兼容	200 Gbps、 120 Mops/s、单 QP 20 Mops、 1 B–2 MB 5.8–8.7 µs 且 P99 ≈ 中位；面积 5–6% / 功耗 3–4%	片上 O(BDP) 缓冲 1–2 mm^2； RPC ULP 未评估



Falcon 硬件 vs Pony Express 软件：操作率 5×、尾延迟 10×（[30] Fig. 1）

规律 1 接口语义比物理带宽更重要。 Dagger 同一链路从 doorbell/DMA 换成缓存一致性共享内存就 +14%，并消除 MMIO 和批处理需求 [12]——CXL.mem/.cache 接入 NIC 的论据；但 CXL 物理层仍是 PCIe， sub-µs 对指针追踪不够 [28]。

规律 2 PCIe 的两个陷阱及解法： 小 DMA 打满事务率 → 设备侧 SRAM 攒一条消息一次性写（+1.78–2.2×）；指针追踪跨 PCIe → 主机先拉平、设备只编码；代价是 CPU 仍参与，节省上限约 74–78% [28]。

规律 3 卸载粒度决定成败： 片外盈亏点 425 B [20]、 Cerebros 每次卸载约 200 周期 [13]、 93% protobuf 消息 ⩽ 512 B [25]—— 小而频繁的任务必须由数据到达方（NIC）自主触发，不能由 CPU 逐次调用。

规律 4 SoC DPU 的 ARM 核不是加速器。 nanoPU （延迟至少 2×）、RpcNIC （编码慢 5.9×）、 FlexiNS （内存带宽 500 vs 需 1,600 Gbps）、BenchBF3 （DPA 单线程更弱）四方一致 [10][28][33][40]。

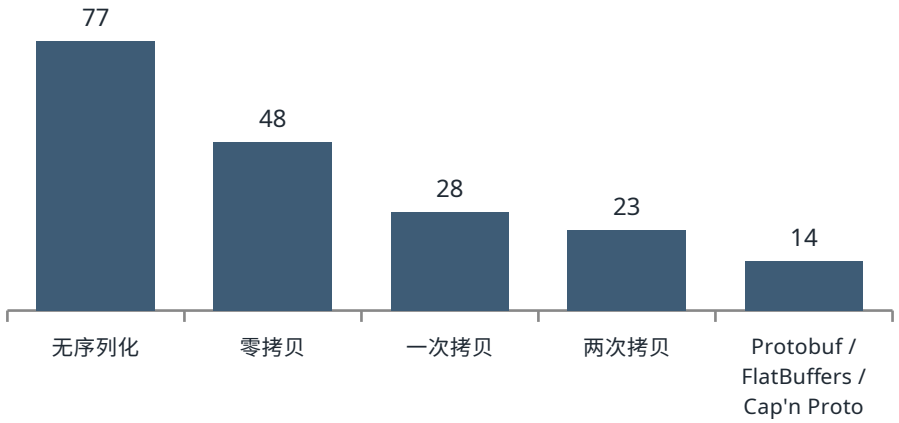
DPU 的现实（2024–25 刻画）： BF-3 DPA 单线程算力 / 访存远弱于主机核、 on-path RDMA 延迟比 off-path 高 12–30%、 DMA 引擎轮询模式某些大小比 BF-2 慢 21–39% [40] → 正确用法是“ARM 决定、硬件搬运”（header-only 卸载 + 主机内存直通 SG），而不是把 RPC 栈整体搬到 ARM 上跑。

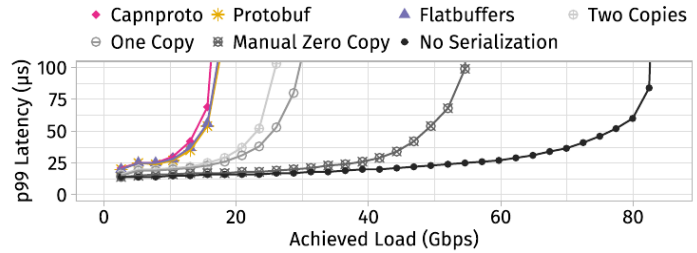
两条路线： Falcon （ASIC 流水线 + ARM 只算慢变量：单 N1 核 20 M events/s 足够） vs BF-3 （ARM / DPA 做数据面）——本调研的证据一边倒支持前者；这直接约束了 UB Controller 与 DPU 的产品定义（第五部分）。

数据来源与口径：表中数字为各论文自报； Cerebros、 µManycore 为模拟， Dagger / nanoPU / RpcNIC 为 FPGA 原型， Falcon 为量产 IPU 实测； [40] BenchBF3 ICNP'24 与 BlueField Idiosyncrasies ICS'25； [20] Accelerometer； [25] ProtoAcc。

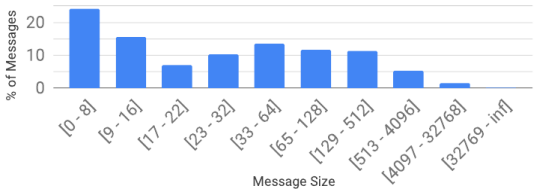
序列化的成本结构：拷贝是主体、元数据其次、编码最少——硬件若只做编码（varint 指令）或只做 DMA，都只能拿到一小块

单核 echo 服务的吞吐（Gbps）：每多一次拷贝掉一档，主流库只剩 1/5（Cornflakes [26]）

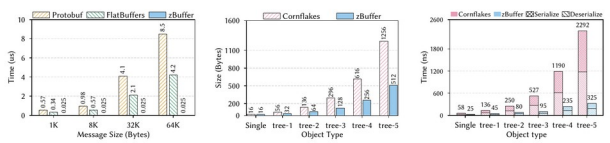




同一实验的 p99 延迟 vs 负载曲线（[26] Fig. 2）



Google fleet protobuf 消息大小：93% ≤512 B （[25] Fig. 3）



阿里 zBuffer：去元数据 + 零拷贝后编解码开销比 Protobuf 低 1–2 个数量级（[19] Fig. 9）

fleet 级画像（Google，ProtoAcc [25]）： protobuf 占 9.6% fleet 周期（C++ 88%）；96% 字节仍是 proto2 → 格式长期稳定；消息极小（24% ≤8 B、56% ≤32 B、93% ≤512 B），但 >32 KB 的 0.08% 消息承载 ≥ 13.7× 于最小桶的字节；按字节 92% 是 bytes / string；小 varint 每字节比大字段慢 100–500×——"没有银弹"；90% 消息只填了 <52% 的字段。

成本分解三份独立实验一致： Cornflakes 单核 echo 77 → 48 → 28 → 23 → 13–15 Gbps [26]；zBuffer：Protobuf 13–14 Gbps → 去元数据 23.4 → 再零拷贝 45 Gbps [19]；zFlatBuffers：缓冲区扩容拷贝占序列化时间 38–58% [40]。Kanev 2015：RPC 税的 2/3 是载荷搬运 [01]；Meta：内存函数最高 37% 周期，memcpy 最大项 [20]。

托管语言更重： Spark 六个应用即使用 Kryo，序列化 / 反序列化仍占平均 28%（最高 83.4%）——对象图递归遍历、反射、GC 头、间接加载叠加 [37]。

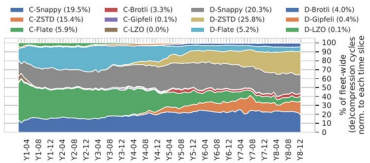
反序列化的复杂度在对象构造与内存分配（ProtoAcc 的 arena、Cerebros 的预留缓冲、zBuffer 的就地修指针、Cereal 反序列化器面积 5× 于序列化器）——与 malloc 税 4.3% 是同一问题。

序列化税 = 拷贝（主体）+ 元数据构造 / 解析 + 编码（varint / TLV）+ 对象分配 / 构造。RPC 卸载若能连带消除请求级小对象分配，收益应把这部分算进去。

数据来源与口径：[26] Cornflakes SOSP'23（单核 echo 微基准，100 Gbps CX-5）；[25] Google MICRO'21 fleet GWP；[19] zBuffer PPoPP'26（阿里，已生产部署）；[40] zFlatBuffers IPCCC'23；[01] Kanev；[20] Meta；[37] Cereal ISCA'20（Spark 六应用）。

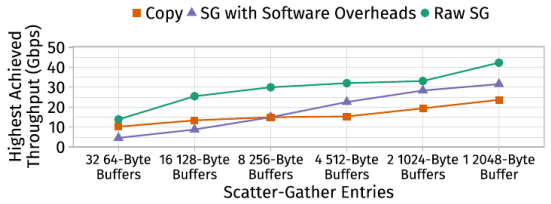
序列化加速器放哪：贴核（覆盖全部 9.6%） vs 片上共享 vs NIC 侧（只覆盖 RPC 路径 1.2%）—— PCIe 附接对中位消息是负收益

方案	放置	覆盖格式 / 语言	面积 / 性能	关键论据
ProtoAcc （MICRO'21） [25]	贴核（RoCC，共享 L2 / TLB，虚拟地址）	protobuf proto2， C++	deser 0.133 + ser 0.278 mm² （22 nm）；比 Xeon 快 3.8×；估省 >2.5% fleet	83% 反序列化不在 RPC 路径； 93% 消息 ≤ 512 B → PCIe NIC 放置不可行
Optimus Prime （ASPLOS'20） [27]	片上共享（NoC 边缘 tile）	通用 DT， Thrift / protobuf	12 流水线 0.45 mm² / 532 mW （28 nm）；约 60× 于软件	PCIe 约 1 µs 对嵌套消息多次往返 → 否决；收益来自预取重叠访存
Cerebros （MICRO'21） [13]	片上 NIC 侧	Thrift compact， C++	RPC 层 − 37–64×；面积比每核复制少 5×	分发必须与序列化同在硬件； NIC 位置可做亲和核选择
RpcNIC （HPCA'25） [28]	PCIe SmartNIC （协同）	protobuf + 可重构内核	主机 CPU −74%；序列化 − 2.3× / −4.3×	主机预序列化拉平 + 设备侧一次性 DMA 规避 PCIe； DSA 帮拷贝
Cereal （ISCA'20） [37]	内存侧加速器（读写 JVM 堆）	Java 对象图（改格式）	2.7 mm² / 1.2 W （40 nm）； Spark S/D 7.97× / 4.81×	反序列化器面积 5× 于序列化器；需与 GC 协作
软件零拷贝： Cornflakes / zBuffer [26][19]	商用 NIC scatter-gather	C++ / Rust，同 ABI	zBuffer 1 KB 编解码 25 ns （Protobuf 1/22.8）； Masstree P99 −21%、吞吐 +62%	裸 SG 64 B 就赢、加簿记后 ≥ 512 B 才赢 → 混合策略；需要 RDMA / DPDK



Google fleet 压缩周期按算法： Snappy / ZStd / Flate 为主（[34] Fig. 1）

压缩：序列化的姊妹问题，且是 RPC 税最大项。 RPC 路径 3.1%、 fleet 2.9%、关键服务 10–50%、存储平台 tax 的 14–31% [02][34][41]。 95% 字节用弱算法 / 低级别以省 CPU → 加速后可换回存储 / 内存 / 网络容量（双重红利）；中位调用 64–128 KiB （比 protobuf 大三个数量级）；文件格式库发起 49% 的压缩周期 [34]。近核 CDPU 64 KB 窗口比 Xeon 快 10×、 0.431 mm² （16 nm）； PCIe 慢 5.6×； chiplet 9.5× [34]。阿里云放弃本地 QAT （<10% 机型、贵 30%、低并发批处理反增延迟）改用远程密钥服务器 [24]—— 片外加速器在云上落地失败的生产案例。



裸 SG 64 B 就赢；加上内存安全簿记后 ≥ 512 B 才赢（[26] Fig. 3）

放置决策链：

- 1. PCIe 附接对中位消息是负收益（盈亏点 425 B [20]、慢 3.4–5.6× [28][34]），除非 RpcNIC 式协同（CPU 仍付 22%）。
- 2. 片上三种放置没有绝对赢家：贴核覆盖全部序列化（含存储 / IPC 的 65–84%）且面积极小；共享硅效率高但受 NUCA 影响； NIC 侧能做整栈卸载但只覆盖网络路径 16–35%。决定因素是客户负载里“网络：存储序列化”的比例——目前没有公开数据。
- 3. chiplet / I/O die 是新的中间选项（CDPU： chiplet 9.5× vs 片上 10× vs PCIe 1.8×） [34]。
- 4. 格式几乎总要改（Optimus Prime 改 setter、 Cereal 改格式、 zBuffer 改布局）；只有 ProtoAcc 保持 wire 兼容。落地障碍是软件接口和生态，不是硅面积（<1 mm²）。
- 5. 调用模型比放置更重要（存储侧）：片上 vs 片外同步只差 1.04×，异步比同步好 1.3×，加速器链式执行（压缩→加密→哈希不回核）≈ 全异步 → “一组可链式调用的小加速器” [41]。

决定序列化加速器放哪的不是硅面积（<1 mm²），而是客户负载里“网络：存储序列化”的比例和软件接口生态；压缩则已有近核方案的定量边界（10× / 0.43 mm²）。

数据来源与口径： [25] ProtoAcc （RTL + FireSim）； [27] Optimus Prime （模拟）； [13] Cerebros （模拟）； [28] RpcNIC （FPGA）； [37] Cereal （模拟）； [26] Cornflakes （实测）； [19] zBuffer （生产）； [34] CDPU ISCA'23 （Google fleet 画像 + RTL）； [41] Google ISCA'23 极限研究； [24] 阿里云 SIGCOMM'24； [20] Accelerometer； [02] Google SOSP'23。

四

综合：热点地图与软硬件发力点

开销落在哪块硅上、量级多少、现有方案能削多少 → 发力点 A （CPU 侧） / B （NIC / DPU / IPU 侧） / C （互连侧） / D （服务器 / SKU 侧）→ 按客户类型的差异 → 争议与开放问题

5.1 表 5-1 （以 Google fleet： library 形态、 C++ 、 TCP 为基准）

热点地图： RPC 相关开销落在哪块硅上、量级多少、现有方案已证明能削多少

开销项	fleet 量级（Google）	热点服务量级	当前落在	已证明的削减手段（比例）	硬件化成熟度
压缩 / 解压	2.9% （RPC 路径 3.1%） [34][02]	10–50% （存储、排序） [34]； Meta Feed1 / Cache1 15% [20]； 大数据平台 tax 的 14–31% [41]	应用核	近核 CDPU 10×、 0.43 mm² [34]； 片外 5.6× 差；加速后可换容量	原型（RTL） + 商用 QAT （片外，云上落地差 [24]）
序列化（全口径）	9.6%，可加速 3.45% [25]	Web 10%、 Feed1 13% [20]； Spark 28% [37]； 大数据平台 tax 的 20–25% [41]； 轻逻辑微服务 40–90% [13]	应用核	贴核 ProtoAcc 3.8–6.2×、 0.41 mm² [25]； 软件零拷贝 zBuffer 13–23× [19]； 大消息多核推测解码 4.9× [42]； PCIe RpcNIC −74% CPU [28]	软件生产（zBuffer）；硬件原型
—— 其中 RPC 路径	1.2% [02] （≈ 0.8% [25]）	Cache 4% [20]	应用核	NIC 侧 Cerebros 37–64× [13]	模拟
RPC 库（分发、路由、加密、观测）	1.1% [02]	Cache2 I/O 52% （含内核） [20]	应用核	Cerebros （与序列化一起）； Google： 单独加速价值不高 [02]	模拟
TCP / 内核网络栈	1.7% [02]； 内核总 ≈ 20% [01]； 大数据平台 system tax 32–42% [41]	Cache 约 30% 时间在内核 [05]； 100 Gbps 双向需 18 核 [32]	应用核 + 内核	用户态 Snap 3×/ 核 [38]； eRPC 10 Mrps/ 核 [31]； Falcon 传输 ≈ 0 核、 IP 占 NIC 5–6% 面积 [30]； CEIO LLC 信用 +1.2–2.9× [33]	Snap / Falcon 生产
memcpy / memmove	4–5% [01] （部分在上两项内）	网络栈拷贝 >50–60% [33]； Ads1 最高 [20]	应用核 + LLC	零拷贝 SG （≥ 512 B） [26]； DSA / I/OAT 卸载 − 55% 主机周期 [28][38]； 片上拷贝加速 +12.7% [20]	DSA 商用
内存分配 / 释放	4.3% [35]	Cache1 最重 [20]	应用核（per-CPU 缓存） + TLB	软件 TCMalloc fleet +1.4% [35]； 硬件卸载只 +1.86% [20]	软件
服务器排队 / 调度 / 唤醒	排队占延迟税 21% [02]； scheduler ≈5% 周期 [01]	Cache 上下文切换 18% [36]； Linux 切换 5K 周期 [36]	内核 + 核	硬件分发 RPCValet +29–76% [11]； nanoPU HTS 尾延迟 − 5× [10]； μManycore −10.4× [36]； Breakwater 过载 <20 ms 收敛 [16]	模拟 / FPGA
跨核负载均衡 / 一致性	每阶段 2–3× [32]	chiplet 跨域 2.07× [35]	NoC / LLC	小一致性域（μManycore）、 NUCA 感知分配器	模拟 / 软件
sidecar 形态附加	Meta 1% 流量	公有云：集群 10% CPU [24]、每 RPC 2.7–3.7× [21] [18]； serverless + 策略 p50 7× / p95 38× [43]	代理核 + 内核	集中网关 − 12–19× [24]； eBPF 内核内 +1.5× [39]； mRPC 6.1× [18]； 轻量沙箱 per-request sidecar （愿景） [43]	部分生产
网络尾延迟 / 拥塞	平均税 1.1%，尾部主导 [02]	—	交换机 / NIC	Homa 100× [08]； Aequitas fleet −10% [17]； Falcon vs RoCE 8×、多路径 180× [30]	生产
前端 i-cache / ITLB	15–30% slot [01]	Web、 Cache [36]	核前端	大 L1-I （DCPerf ARM SKU 反例 − 85% Perf/W） [05]； Cerebros 卸 RPC 层 MPKI −93% [13]	微架构
IOMMU	—	网络栈 − 26% [33]	IOMMU / IOTLB	ATS / 大页 DMA	微架构

量级排序（fleet 平均）： 内核 ~20% （不属可加速税） > 序列化全口径 9.6% > malloc 4.3% ≈ memcpy 4–5% > 压缩 2.9% > TCP 栈 1.7% > RPC 库 1.1%。只看 RPC 路径：压缩 3.1% > TCP 1.7% > 序列化 1.2% > RPC 库 1.1%。
热点服务排序： 缓存 / KV 服务的 RPC I/O + 内核 + 切换（52% + 30% + 18%）；轻逻辑微服务的 RPC 层（40–90%）；大数据的序列化（28%）。
硅位置排序： 绝大多数开销落在应用核上（这就是为什么 Meta 单 SKU、 Perf/W 决定采购）；其次是内核 / LLC / NoC； NIC / 交换机只承担网络尾延迟。

数据来源与口径：“可削减”指现有论文已证明的削减比例；“位置”指该开销当前主要消耗哪块硬件；引用编号见附录。

发力点建议：按“谁能做、收益多大、成熟度如何”分为 CPU 侧、 NIC / DPU / IPU 侧、互连侧、服务器 / SKU 侧四组

A CPU 侧（芯片厂商可独立推进）

A1 近核“数据变换加速器簇”：序列化 + 压缩 + memcpy。三者合计 fleet 15–20%； ProtoAcc 0.41 mm² 覆盖 3.45%、 CDPU 0.43 mm² 覆盖 2.9% 并换回容量；必须贴核 / 片上；异步 + 链式调用；收益按 zBuffer 而非 Protobuf 基线计算。

A2 片上 DMA / 拷贝引擎的用户态低延迟提交与小段优化。多数拷贝 <512 B，小段固定开销要 <100 ns；与 NIC SG 协同。

A3 前端：大 L1-I / ITLB、切换感知的指令 cache 管理，而非更聪明的顺序预取器；替换策略可调（FB Web +2.9%）。

A4 硬件请求调度与上下文切换：消息驱动的核唤醒、硬件线程队列与优先级抢占（nanoPU −5×、 μManycore −10.4×）。

A5 LLC 作为 NIC↔CPU 缓冲的主动管理：可配置 DDIO way、NIC→LLC 信用、按队列 cache 分配（CEIO +1.2–2.9×）。

A6 chiplet / NUCA 感知：暴露 cache 域拓扑、可动态划分一致性域（跨域 2.07×）。

A7 IOMMU / ATS：− 26% 的虚拟化税； ATS / PRI、大页 DMA、更大 IOTLB。

A8 内存保护 key 与 CXL 共享内存安全：更多 key、用户态可控的页 / 对象级权限切换（RPCool 0.35 μs 沙箱）。

A9 不值得单独硬件化： malloc（硬件只 +1.86%）、 RPC 库本身、内核——靠软件与微架构通用改进。

B NIC / DPU / IPU 侧

B1 消息式硬件传输（Falcon 功能清单）：延迟型 CC + 硬件时间戳 + 接收侧缓冲占用回传、位图 SACK、拥塞感知多路径 + 片上 O(BDP) 缓冲（1–2 mm²）、请求–响应事务接口、不依赖无损网络； 18 核 → ≈ 0 核。

B2 按实时核负载的动态分发（替代 RSS）： RSS 浪费 30–76% 吞吐；需要 NI↔核 ns 级反馈——片上 / 封装内 NI 才能做，是 CPU 厂商的结构性优势。

B3 SG / DMA 小段开销与接收侧 SG：条目数上限 ~30 → 更多、每条目 <100 ns、设备侧按消息聚合写回、接收侧按 schema 分散落点。

B4 RPC 边界与优先级元数据保留： Aequitas 需要按 RPC 而非连接标 QoS。

B5 L7 解析结果作为元数据交付（“L7 RSS”）：解析占 sidecar 开销 63–77% 且应用又解析一遍；但跳数和拷贝才是主体， DPU 搬走代理并不自动省钱。

B6 ARM / DPA 只做控制面与包头， payload 走硬件直通。

B7 加密加速要么标配要么别做：价值在连接建立和 sidecar / 网关形态，需要低延迟小批量路径。

C 互连侧（CXL）

C1 CXL 对 RPC 的价值在接口语义和机架内共享，不在延迟量级。 Dagger +14%、 HydraRPC 1.47 μs；但 DB 主导的微服务无收益。

C2 需要的控制器 / CPU 功能：廉价的非缓存共享、内存写触发的轻量通知（当前只能轮询）、按主机的访问窗口 / 租约 / 回收、多主机内存保护。

C3 放置策略随之改变：同 CXL 域内服务对用共享内存 RPC（1.5 vs 17 μs）→ 调度器把高频通信服务放进同一域； CXL 挂旧 DRAM 省碳。

D 服务器 / SKU 侧

D1 单一 SKU 约束 + Perf/W + 碳： Meta 非 AI 单 SKU； DCPerf 靠 Perf/W 选中绝对性能更低的 ARM SKU；“同一芯片、多运行模式”比多 SKU 更现实。

D2 内存：带宽过配、延迟与热页优先。利用率 10–31% → 通道换容量 / CXL 扩展 + 小容量高带宽层。

D3 拓扑： NIC 位置与 NUMA。 NIC 远端 − 20%；多 die 下 NIC 挂载位置影响 RPC 路径。

D4 用真实负载做设计输入并交付内核调优：DCPerf 指导 ~10 项优化 FB Web +38% / Perf/W +47%， SPEC 完全看不出； 384 核内核锁争用 54%。

一句话： CPU 侧以 fleet 加权的序列化 + 压缩 + memcpy 簇（A1 / A2）和前端 / 切换效率（A3 / A4）为主目标； NIC 侧以 Falcon 式消息传输（B1）和动态分发（B2）为主；存储 / KV 前端是专用加速的落点。

数据来源与口径：每条建议的依据数字见 REPORT.md 5.2 节与本 PPT 对应页；编号 A1–D4 在第五部分鲲鹏映射中继续使用。

同一个发力点对不同客户价值不同： hyperscaler 、公有云租户、 Java / 大数据、存储 / KV 前端、CXL 机架采用者

客户类型	RPC 形态	RPC 税量级	最相关的发力点
hyperscaler （Google / Meta 型）	library （99%）、 C++ / Hack 、自研传输（Snap → Falcon ）	fleet 7% （RPC 路径） + 序列化全口径 9.6% + 压缩 2.9%	A1 （近核序列化 / 压缩簇）、 A3 （前端）、 B1 （Falcon 类传输）、 B2 （动态分发）；评估工具 DCPerf / HyperProtoBench
公有云租户 / 企业（Istio 型）	sidecar （2.7–3.7× ）→ Ambient / 集中网关 / eBPF ； serverless + 策略 7–38×	集群 10–30% CPU 在 mesh 上； FaaS 每节点 +2.5–6.5 核	B5 （L7 元数据）、 B7 （加密）、集中网关的 DPU / 加速； XLB 类内核内 L7 需要的是核内效率而非卸载
Java / 大数据（阿里、 Spark ）	容器混部、 RPC 76% / MQ 23% 、 JVM	序列化 28% （Spark ）； CPU 干扰敏感	Cereal 类 JVM 对象序列化加速（空白市场）、硬件隔离（核 / LLC / 带宽 QoS ）、快速唤醒（利用率 <10% ）
存储 / KV 前端（RPC 服务端主体）	高 QPS 、 µs 级、内核重	I/O 52% + 内核 30% + 切换 18%	A4 （硬件调度 / 切换）、 A5 （LLC ）、 B2 （分发）、 B1 ； nanoPU / µManycore 路线的目标客户
CXL 机架架构采用者	同机架共享内存 RPC	通信 + 拷贝 + 序列化三项在域内接近消失	C1–C3

争议与开放问题（也是芯片厂商可以用第一手数据回答的）

1 加速器瞄 fleet 平均还是热点服务？ Google 说 RPC 库加速价值不高、序列化 / 压缩更值得 [02]； Cerebros 说 RPC 层占 40–90% [13] 。两者都对——建议以 fleet 加权的序列化 + 压缩 + memcpy 为主目标（A1 / A2 ），以存储 / KV 前端为专用加速落点（A4 / B2 ）。

2 序列化加速器放哪？ 贴核（覆盖全部 9.6% ） vs NIC 侧（只覆盖 1.2% 但能整栈）——取决于客户负载的网络 ：存储序列化比例，目前没有公开的 fleet 级数据（第一个空白）。

3 CXL 一致性要不要？ HydraRPC 绕开 CXL.cache 并断言其开销不可容忍 [14]； RPCool 假设一致性可用 [29]； Dagger 从一致性互连获益 14% [12] 。真实 CXL 3.x 多主机一致性硬件的实测尚无。

4 DPU 的定位： Falcon （ASIC 流水线 + ARM 控制面） vs BF-3 （ARM / DPA 数据面）；证据一边倒支持前者，但 FlexiNS 证明后者在正确的编程模型下也能到 400 GbE 线速 [33] 。

5 零拷贝对小消息无效（MeshInsight 、 Cornflakes 、 Canal eBPF 小包）——中位 RPC <1.5 KB ，小消息的出路是减少跳数和批量；硬件应提供“消息驱动的批量 DMA”。

6 / 7 两个数据空白：托管语言的 RPC / 序列化税几乎没有 fleet 级数据（只有 Spark 的 28% ）；部署形态的 fleet 占比在任何公司都未公开——三角验证无法闭合的原因。

验证工具： DCPerf （5 类负载 + tax 微基准）、 HyperProtoBench 、 HyperCompressBench 、 DeathStarBench ；收益预测 = Accelerometer （卸载上限） × MeshInsight （按调用图放大） × CRISP （关键路径决定延迟收益）。

数据来源与口径： REPORT.md 5.3–5.5 节；表中量级来自第一、二部分对应页；争议各方引用见括号。

五

面向鲲鹏产品线的映射： CPU / KAE / UB

方法与对照系（Intel DSA / IAA / DLB / QAT）→ 鲲鹏 CPU （前端、一致性域、 SMT 、 LLC 、 IOMMU ）→ KAE 与 DPA （压缩、加密、序列化空白、 DPA 作 DSA ）→ UB 对标 RDMA / CXL → 优先级与“待测”清单

方法与对照系：把第五章的建议写成“文献要什么 → 鲲鹏现状 → 差距 → 怎么验证”，并用 Intel 四类片上加速器作坐标系

方法与依据的可信度

三个对象：鲲鹏 CPU、KAE 加速器、UB（灵衢）互连。每条映射写成“文献说需要什么 → 目前有什么（公开可查）→ 差距或机会 → 怎么验证”。

依据分三类，可信度不同：

- 文献：第 1–5 章的论文结论，引用 [NN]。
- 〔公开〕：hikunpeng.com、KAE 仓库（gitcode.com/boostkit/KAE）、openEuler UADK 文档、华为 HC2025 发布（鲲鹏 950 / 960、TaiShan 950 SuperPoD、UB 2.0）、Chips and Cheese 对鲲鹏 920 的微架构测试。
- 〔本地〕：~/ub_doc_and_code（概念手册、内存语义与 URMA 分析、软件栈分析、UMQ 机制分析）、~/umdk、~/ubs-comm（HCOM / ubsocket）、~/brpc_ub（bRPC over UB 适配）。

没有的东西：鲲鹏 920 / 950 的内部微架构参数、KAE 引擎的调用延迟、UB 硬件的小消息时延 / Mops—— 本部分反复标注“待测”，汇总在第 36 页。

本部分的价值是把文献的定量结论翻译成“在鲲鹏上应该先测什么、先做什么”，而不是替代内部数据。

对照系：Intel 四类片上加速器 vs 鲲鹏

Intel 片上加速器	功能	对应第五章建议	鲲鹏公开产品中的对应物	状态
DSA	拷贝 / 数据搬移、填充、CRC、比较	A2 片上 DMA 引擎的小段低延迟提交	鲲鹏片上的 DPA（数据搬移加速器）〔内部口径〕	对应，提交延迟待测
IAA	压缩 / 解压 + 内存分析（扫描、过滤）	A1 数据变换簇中的压缩与“数据变换”	KAE ZIP 覆盖压缩；“数据变换 / 序列化”部分无对应	压缩有；数据变换空白
DLB	硬件队列与负载均衡（核间任务分发）	A4 硬件请求调度 / B2 动态分发	NI 侧有 UB Jetty Group（RQ 深度分发）；核侧软件间通用硬件队列无对应	核侧空白
QAT	加密 + 压缩	B7 加密标配 / A1 压缩	KAE SEC / HPRE / ZIP（片上共享引擎，免许可）	已覆盖

结构性结论：KAE 目前覆盖 QAT 的领地（压缩 + 加密）；数据搬移由片上的 DPA 负责，对应 DSA；IAA 的“数据变换”部分和 DLB 的“硬件队列 / 分发”在鲲鹏公开产品中还没有对应物。

与文献量级对照：序列化全口径 9.6% 是压缩 2.9% 的 3 倍，是 KAE 相对 IAA / DSA 的最大空白；核侧硬件队列 / 切换（A4）的文献收益上限最高（nanoPU −5×、μManycore −10.4×），是整个 ARM 服务器阵营相对 Intel 的空白。

一个坐标系看清结构：压缩 / 加密（KAE）已有，拷贝引擎（DPA）的软件接口待打通，序列化 / 数据变换与核侧硬件队列是空白——后面四页按这个顺序展开。

数据来源与口径：Intel DSA / IAA / DLB / QAT：Intel Sapphire Rapids 公开资料；鲲鹏对应物来自 KAE 仓库、openEuler 文档与本地 UB 文档 / 代码（REPORT.md 6.0、6.5 节）。

鲲鹏 CPU： 920 公开可见的三个微架构弱点（L3 取指带宽、小 BTB、慢 L2 TLB）恰好命中微服务文献排第一的热点——前端

第五章建议	文献要点	鲲鹏现状〔公开〕	差距 / 机会	验证方法
A3 前端	前端 stall 15–30% [01]；生产 L1-I MPKI 7–56、DCPerf 里一颗 ARM SKU 因 L1-I 只有 1/4 被淘汰 [05]；顺序预取 / 更聪明的分支预测对微服务无效，足迹来自服务间 / OS 切换 [36]	920： L1I 64 KB 够大，但 L3 取指带宽差（“接近从 DRAM 取指”）、64 项单周期 BTB、L2 TLB 11 周期；950 LinxiCore 未公开	微服务的指令足迹落在 L2 / L3 上，L3→L1I 取指路径的带宽 / 延迟是 920 上最直接命中文献热点的弱项； BTB 容量与多级 BTB 对大代码库（Web 类）重要	在 920 / 950 上跑 DCPerf 五类负载，采 L1I / L2 / L3 code-fetch MPKI、ITLB miss、前端 bound 占比；对比 Graviton 3/4、Ampere
A6 chiplet / NUCA	跨 LLC 域 2.07× [35]；跨核网络栈 2–3× [32]；小一致性域主张 [36]； 384 核内核锁争用 54% [05]	920 多 compute die、每 die 一 NUMA、L3 分 bank（tag 在 cluster 侧）； 950 192 核 / 384 线程	核数翻倍后“可划分一致性域 / 向软件暴露 cache 域拓扑”比再堆核更贴近微服务需求；内核可扩展性（锁、per-CPU、RCU）要在 384 线程上提前验证	同 die 内 vs 跨 die 的 client-server RPC 往返延迟与吞吐；内核 syscall / 网络栈随核数的扩展曲线
A4 唤醒 / 切换 + SMT	Linux 切换 5K 周期、软件调度 5 µs 粒度对 µs 任务无效 [10][36]； Cache 服务切换占 18% CPU [36]； >50 µs 唤醒与尾延迟强相关 [02]； SDT 专用投递线程面积 − 47.5% / 功耗 − 66% [40]	950 引入 SMT （LinxiCore 双线程）； ARM 有 WFE / WFET （FEAT_WFxT）可对应 x86 umwait	SMT 是双刃剑：兄弟线程干扰伤尾延迟，但“把网络投递放在专用硬件线程”是 SMT 的正确用法 → LinxiCore 若能提供投递线程优先级 / 资源隔离就是差异化点；缺核侧硬件线程队列与切换（nanoPU HTS、µManycore）和 DLB 式硬件队列	测 SMT 开 / 关下 KV 类服务的 P99；测 WFE → 唤醒延迟与 C-state 退出延迟；用 Accelerometer 模型 + µManycore 数据评估硬件队列 IP 的收益上限
A5 LLC 作 NIC↔CPU 缓冲	DDIO 2 way vs BDP → 49% miss、− 24% 吞吐 / 核 [33]；CEIO 信用 = LLC 容量 +1.2–2.9× [33]	920 SoC 集成 100 GE NIC （hns3）； ARM CHI 有 cache stashing；鲲鹏 NIC / UB Controller 是否直写 L3、way 数是否可控：待查	若 UB Controller 直写 L3 且可按队列控制容量，就能实现 CEIO 的“信用 = LLC 容量”—— UB Controller 在 SoC 内相对 PCIe NIC 的天然优势	随 BDP 增大测 LLC miss 率与每核吞吐；对比 UB 直通与 PCIe RoCE
A7 IOMMU / SVA； A8 内存保护 key	IOMMU 开启网络栈 − 26% [33]； RPCool 依赖 MPK （16 key、0.35 µs 沙箱） [29]	SMMUv3 + SVA 是 UADK 与 UB （UMMU non-pin、USVA）的基础； UB 的 UMMU 是独立于 SMMU 的第二套地址翻译 / 权限硬件〔本地〕； ARM 无 MPK，Armv8.9 的 POE 提供用户态可切换页权限； 950 四层隔离细节未公开	UMMU 的 TLB / PLB 命中率和 miss 代价将直接进入每 RPC 成本，虚拟化下（Stage2 + vfio-ub）尤甚；若 LinxiCore 支持 POE， UB 共享内存 RPC 就能做 RPCool 式 sub-µs 沙箱	测 UMMU TLB miss 对小消息延迟的影响（大页 vs 4 KB）； SMMU 开 / 关对 hns3 / UB 的吞吐差； POE / MAPT 权限切换延迟对照 0.35 µs
D1 / D2 / D4 Perf/W、内存、真实负载	Meta 单一 SKU、power 是最紧约束 [22]； DCPerf 靠 Perf/W 选中绝对性能更低的 ARM SKU [05]；带宽利用率 10–31% 但热页集中 [01][05][40]； SPEC 调优看不到 FB Web +38% [05]	950 一颗芯片两款核数； 12 通道 DDR5 （二手来源，待核实）； TaiShan 950 SuperPoD 内存池化（16 节点 48 TB，虚拟化内存利用率 +20%）； Chips and Cheese 对 920 的批评是 SPEC 口径	“同一芯片、多运行模式”（可划分一致性域、可开关硬件调度、可分区 LLC）比多 SKU 更现实；通道换容量 / UB OBMM 池化换利用率；如果用 DCPerf / HyperProtoBench 而非 SPEC 调优 950 / 960，可在对手不擅长的口径上建立优势	把 DCPerf Perf/W 口径纳入内部回归； MemProf 类方法测热页集中度； UB 远端内存作“低带宽层”对 DCPerf 的影响

结论 1： 920 的三个公开弱点（L3 取指带宽、小 BTB、慢 L2 TLB）正是微服务文献中排第一的热点（前端），比任何加速器都更影响 fleet 平均； 950 LinxiCore 是否补上，是第一个要测的问题。

结论 2： 192 核 + SMT 的 950 面向密度，而微服务的问题是每核效率与尾延迟；把核数优势变成客户价值需要一致性域可划分、内核可扩展性验证、SMT 投递线程隔离三件配套。

结论 3： 核侧硬件队列 / 切换（DLB 类）是鲲鹏（和整个 ARM 服务器阵营）相对 Intel 的空白，文献收益上限很高（nanoPU −5×、µManycore −10.4×）； NI 侧的分发 UB Jetty Group 已具备，两者合起来才是 RPCValet → nanoPU 的完整路径。

数据来源与口径：鲲鹏 920 微架构： Chips and Cheese 测试〔公开〕； 950 / 960： 华为 HC2025 〔公开〕， 950 的 L3 / 内存通道数据为二手报道待核实； UMMU / USVA： 本地 UB 文档〔本地〕；文献引用见括号。

KAE： 架构位置对“ KB 级同步压缩”够、对“百字节级序列化”不够——三件事：压缩的小 buffer 开销、 DPA 承担 DSA 角色、加密聚焦网关与国密

KAE 公开现状〔公开〕： ZIP （ zlib / gzip 、 zstd 、 lz4 、 snappy ； 单处理器最高 7 GB/s 压缩、 8 GB/s 解压）、 SEC （ AES 、 SM4 、 SM3 、 MD5 ）、 HPRE （ RSA 、 DH 、 SM2 ）； 接入 OpenSSL engine / Tongsuo / BoringSSL / zlib 兼容 API / UADK （ uacce + SVA ，队列式提交，部分算法异步）； SoC 片上共享引擎（非 PCIe 卡），队列 SEC 256 / HPRE 、 ZIP 各 64 ； 920 新款免许可、 950 支持。

在“放置之争”坐标上： KAE 是片上共享型——比 PCIe / QAT 卡近（ CDPU 测得 PCIe 慢 5.6× [34] ），比 ProtoAcc / CDPU 的“贴核”远（它们要求 ones-of-cycles 协处理器接口和共享 L2 ），接近 CDPU 的 chiplet 情形（ 9.5× vs 贴核 10× ，可接受）。

开销项（表 5-1 ）	fleet 量级	KAE 现状	差距 / 机会	建议
压缩 / 解压	2.9% （ RPC 路径 3.1% ）； 热点服务 10–50% ； Meta Feed1 / Cache1 15%	算法全覆盖； 7 / 8 GB/s 是大块吞吐口径	WSC 压缩的特征是小 buffer 、同步调用、弱算法（ 95% 字节）——决定收益的是 KB 级 buffer 的固定开销（提交、完成、 SVA 翻译）而非峰值带宽； CDPU 近核 10× 、 PCIe 1.8× ，片上共享介于两者之间； “加速后换更强算法 → 换回容量”是第二条价值线 [34]	(1) 测 1 / 4 / 16 / 64 KB buffer 的盈亏粒度，目标 4 KB 同步往返 1–2 µs 以内； (2) 在 gRPC / bRPC / HCOM 压缩钩子和 Protobuf payload 压缩点透明接入； (3) 用 HyperCompressBench 报告“省 cycles”和“换容量”两个口径
加密	Google <0.2% （连接复用）； TLS 建连 1.6 ms / 14 KB [21] ； 阿里云弃用 QAT [24]	SEC / HPRE 覆盖握手与对称； OpenSSL 接入成熟；片上、免许可	KAE 正好回应 B7 ： 片上 + 免许可消除了阿里云弃用 QAT 的两个理由（机型覆盖、成本）； 剩下“低并发批处理增延迟”需要低延迟小批量路径；价值落点是 sidecar / 网关和连接建立，不是 fleet 数据面	测 Envoy / Nginx + KAE 在 1–8 连接低并发下的握手延迟是否劣于软件； 提供“单请求同步、不攒批”模式；国密（ SM2 / SM3 / SM4 ）作国内差异化标配
序列化（ KAE 空白）	全口径 9.6% 、可加速 3.45% [25] ； RPC 路径 1.2% ； Spark 28% [37]	无	比压缩大 3 倍的项，也是相对 IAA / DSA 最大的空白。但不能用 KAE 现有队列模型做：调用粒度百字节到 KB （中位 RPC <1.5 KB ）、 65–84% 在存储 / IPC 路径， ProtoAcc / Optimus Prime 都要求贴核	两条并行路线：软件—— zBuffer 类 SG 反射直接落到 UB 的 SGE （ UMQ qbuf 池已是 DMA-ready 内存），不需要 KAE ，收益 13–23× ； 硬件——由 LinxiCore 团队做近核“数据变换微引擎”， KAE 承担 schema 编译器与运行时接口；收益按 zBuffer 基线计算
memcpy / DMA	4–5% ； 网络栈拷贝 >50–60% ； RpcNIC 用 DSA −55% 主机周期 [28] ； 多数拷贝 <512 B	KAE 不做搬移；片上数据搬移由 DPA 负责〔内部口径〕	DPA 是鲲鹏对应 DSA 的部件；文献要求小段（ 64 B–4 KB ）固定开销 <100 ns 、 SG 批量描述符、与 NIC SG 协同； DPA 提交路径与小段延迟未知	测 DPA 用户态提交固定开销与盈亏粒度；若 >1 µs 则需用户态 doorbell 式提交；与 UB 内存池协同
hash / CRC ； malloc 、 RPC 库、内核	hash 1–2% ； 4.3% / 1.1% / ~20%	SM3 / MD5 在 SEC ； HCOM 有硬件 CRC 选项； ARM CRC32 指令	A9 ： 文献明确不建议单独硬件化（ malloc 硬件卸载只 +1.86% ， RPC 库加速价值不高）	保持指令级实现；不投 KAE 资源

接口模型是第二个战场： 同步 vs 异步 vs 链式——片上 vs 片外同步只差 1.04× ，异步比同步好 1.3× ，加速器链式执行（ ZIP → SEC → 哈希不回核）≈ 全异步，同步模式下 setup 时间造成大幅退化 [41] 。 KAE 三个引擎能否共享描述符 / 缓冲直接链式传递，比单引擎峰值带宽更重要； UADK 异步要对接 bRPC bthread 、 gRPC completion queue 、 Java 虚拟线程——建议提供“协程友好”的完成通知（ eventfd / JFCE 类，与 UB 完成事件通道复用）。

数据来源与口径： KAE： gitcode.com/boostkit/KAE 、 openEuler UADK 文档〔公开〕； UMQ： 本地 UB 文档与代码〔本地〕； DPA： 内部口径（ 2026-09-29 ）； 文献 [34] CDPU 、 [41] Google ISCA'23 、 [25] ProtoAcc 、 [19] zBuffer 、 [28] RpcNIC 、 [24] 阿里云、 [21] ServiceRouter 、 [20] Accelerometer 。

UB 对标 RDMA / CXL ： 传输层已具备文献要求的大部分特性（无连接可靠、松弛排序、多路径、用户态权限、事件通知、 DWQE ），缺口在“ RPC 感知”和“硬件分发”

文献需求	为什么重要	UB 对应机制〔本地〕	状态	与 RDMA / CXL 的相对位置
消息语义 + 每消息极小状态，避免连接 cache 悬崖	RDMA RC 的 QP 状态随连接数溢出 NIC cache [31][38]；每 client 连数万后端 [21]	RM 模式无连接：每条 WR 自带目标 EID / Jetty； TP Channel 按节点对共享（O(节点²)）； MUE 托管上下文	有	优于 RoCE RC，接近 Homa / Falcon ；万节点下 TP 表的片上缓存行为待测
松弛排序 + 拥塞感知多路径（ Falcon vs RoCE 8× 的来源）	RoCE 单路径 + 严格有序是尾延迟主因 [30]	ROI / ROT / ROL / UNO 可选； TPG ≤32 通道 + 逐包 spraying + 接收端 OOR 重排	有	与 Falcon 同级； Falcon 的预算：接收侧 O(BDP) 缓冲 1–2 mm² 是不出悬崖的前提
不依赖无损网络（端到端重传、 SACK ）	PFC 风暴是 RoCE 部署痛点	RTP 端到端 PSN 重传 + CC ； CTP 依赖链路层无损（机架内低时延）	有	两档可选：数据中心用 RTP，超节点内用 CTP 拿时延
延迟型 CC + 硬件时间戳、端主机拥塞信号	Falcon ： 四时间戳 fcwnd + 接收 NIC Rx 缓冲占用回传 ncwnd [30]； RNL 测量需时间戳 [02]	传输层 CC （窗口 / 速率 / CAQM ）；链路层 VL 信用；事务层 RNR ； SL 0–15	部分	CC 是否可编程、 CQE 是否带硬件时间戳、有无接收侧资源占用回传：待测
按 RPC （非按连接）的 QoS 标记	Aequitas fleet −10% 依赖按 RPC 标 QoS [17]	SL / VL 存在； ubsocket 当前是每链路一个 link_priority	部分	差距：需要 RPC 库把每消息优先级写进 WR → SL ； UB 比 TCP 容易（消息边界 = WR 边界）
接收端驱动 / 授信	Homa grant [08]、 Breakwater 信用 [16]	UMQ 应用层信用（立即数内授信、 FC Jetty 、共享信用池）；事务层 RNR 兜底	有（软件层）	硬件层没有 receiver-driven grant
用户态权限 / 免注册	Cornflakes 簿记根源于 NIC 只能 DMA pinned 页 [26]	UMMU MAPT 可下放（ PERMQ ）、 Token 可转授、 non-pin SVA 段； UMQ 整池注册一次	有	优于 RDMA （ MR 注册走内核）；接近 CXL 但多细粒度权限
按核实时负载的硬件分发（替代 RSS ）	RSS 浪费 30–76% 吞吐 [11]； nanoPU JBSQ [10]； Cerebros 函数亲和 [13]	Jetty Group ： 多 RQ ，硬件按 Hint 哈希 / RR / RQ 深度自动选 RQ ， NUMA 亲和；共享 JFR	有（ RQ 深度）	已是 RPCValet 式 NI 侧动态分发；与 nanoPU 的差距在有界队列 + 超时回收 / 抢占、能否按 RPC 方法分发：待测
写触发的轻量通知；免 doorbell 提交	CXL 只能轮询 [14]； >50 µs 唤醒伤尾延迟 [02]； Dagger 内存互连 +14% [12]	JFCE 完成事件、消息中断、远端事件通知； DWQE 用 ARM64 st64b 把整条 SQE 直写设备	有	优于 CXL 2.0 ； SoC 内无 PCIe 往返是结构性优势，需小消息 Mops/ 核 数据证明
域内共享内存 RPC ；可控一致性；内存池化	C1–C3 ： 1.5 vs 17 µs [29]； HydraRPC 绕开 CXL.cache [14]；带宽分层 [40]	OBMM 共享 + Ownership ； Load/Store 走 TP Bypass ；借用 / 共享 / 非缓存三档； UBFM / UBPRM 租约回收； NUMA_REMOTE 上线	有	域比 CXL 大（ 16 节点 / 200 m ）；走的正是“软件一致性共享 + 大域”路线；缺 MPK 类沙箱和公开的远端访存延迟

bRPC over UB ： 已落地的“形态 A” 样本

- brpc_ub = bRPC + ubsocket ： epoll / readv / writev 全部改走 ubsocket_wrapper_* ； TCP 握手交换 EID / Jetty 后切到 UMQ ，失败逃生回 TCP ； IOBuf 分配器 hook 到 UMQ 注册内存池， readv / writev 是“ Block 链表挂接”的零拷贝协议〔本地〕。按第二部分分类：进程内库 + 用户态 UB 栈，对应 Snap / eRPC / Junction 的格子。四个评估点：

1 连接扩展性： 默认 RC_TP 一对一绑定（“同 IB RC 的扩展性问题”），应测 RM_TP 在数千对端下延迟悬崖是否消失——这是 UB 相对 RoCE 的核心卖点。

2 小消息效率： 中位 RPC <1.5 KB ，而 ubsocket 最小块 8 KB 、 qbuf 最小 4 KB slab—— 需要小消息 slab 或 inline （ Falcon / eRPC 对 <256 B 走 inline ）。

3 序列化接缝： bRPC 仍是 Protobuf → IOBuf 拷贝式； zBuffer 的 SG 反射可让字段直接成为 UMQ 的 SGE ，叠加 13–23× 收益——软件侧最便宜的一步。

4 建连与逃生成本： TCP 握手 + UB 协商是每连接固定开销，短连接 / serverless 下放大； UM 模式带内自举能否替代 TCP 握手值得评估。

最需要的是数据： 文献里每个方案都有“每核 Mops 、 RTT 、每 Gbps 核数、连接数 vs 延迟”四个数， UB 目前公开的只有超节点级 2.1 µs 和 HCOM 软件时延 ≤ 0.5 µs 目标。用 eRPC （ 2.3 µs 、 10 Mrps/ 核）、 Falcon （ 120 Mops ）、 Homa/Linux （ 18 核 @100 Gbps ）作对标口径测一遍，是 UB 在 RPC 场景立论的第一步。

数据来源与口径： UB 机制：本地 ~/ub_doc_and_code 、 ~/umdk 、 ~/ubs-comm 、 ~/brpc_ub 〔本地〕； UB 2.0 时延 / 带宽：华为 HC2025 〔公开〕；文献需求清单来自 B1 （ Falcon ）、 Homa 、 eRPC / Snap 、 Aequitas 、 RPCValet / nanoPU 、 Dagger 、 HydraRPC / RPCool 、 Cornflakes 。

面向鲲鹏的优先级：先建基线（DCPerf 与 UB 小消息），再补前端 / 一致性域，然后是 KAE 两个扩展、UB 的 RPC 感知、zBuffer 落 UB

优先级	事项	对应建议	类型	为什么排这里
1	在鲲鹏 920 / 950 上建立 DCPerf / HyperProtoBench / HyperCompressBench 基线，得到鲲鹏自己的 tax 分解（前端 MPKI、内核占比、序列化 / 压缩 / memcpy 周期、SMT 干扰）	D4	测量	后续所有判断的依据；SPEC 口径看不到这些（FB Web +38% 在 SPEC 上完全不可见 [05]）
2	UB 小消息 RPC 基线：RTT、Mops/ 核、每 Gbps 核数、连接数（RC vs RM） vs 延迟、通知 → 唤醒延迟、Load/Store 远端访存延迟	B1 / C1	测量	UB 在 RPC 场景的优势都是结构性推断，需要与 eRPC / Falcon / HydraRPC 同口径的数据
3	LinxiCore 前端与一致性域：L3 取指路径、BTB、L2 TLB；可划分一致性域；内核 384 线程可扩展性	A3 / A6 / D1	微架构	fleet 平均最大项（前端 15–30%）；920 的公开弱项正在这里
4	片上加速器两件事：KAE 压缩小 buffer 固定开销 + RPC 库接入；DPA 做用户态可用的 DSA（<100 ns 小段、SG、与 UB 内存池协同）	A1 / A2	加速器	压缩 2.9% + memcpy 4–5%；两者都是文献已验证、KAE 架构位置够用的项
5	UB 的 RPC 感知增强：每消息 SL 映射、CQE 硬件时间戳、可编程 CC、Jetty Group 有界队列 / 按方法分发、bRPC-UB 默认 RM + 小消息 inline	B2 / B4 / A4	互连 + 软件	文献收益上限高（分发 +29–76%、Aequitas −10%），且是 UB Controller 在 SoC 内的结构性机会
6	zBuffer 类零拷贝序列化落到 UB SGE（bRPC / HCOM / gRPC 三条线）	A1 软件路线	软件	最便宜、收益已在阿里生产验证（13–23×）[19]；不依赖新硬件
7	超节点内共享内存 RPC（HydraRPC / RPCool 在 UB OBMM 上复现） + POE / MAPT 沙箱	C1–C3 / A8	互连 + CPU	一个数量级的延迟差距 [29]，但只对同域高频通信服务对有效，需调度器配合
8	近核序列化微引擎（ProtoAcc 类，LinxiCore 侧）	A1 硬件路线	微架构	量级最大（9.6%）但生态与接口风险最高，收益要按 zBuffer 基线折算

“待测”清单（需要内部数据的项）

CPU： 950 L1I / L2 / L3 取指带宽与 BTB 规模；SMT 兄弟线程对 P99 的影响；WFE → 唤醒与 C-state 退出延迟；POE 是否可用及 MAPT 权限切换延迟。

内存 / IOMMU： UB Controller / NIC 是否直写 L3 及容量控制；UMMU TLB / PLB miss 代价（大页 vs 4 KB）；SMMU 开 / 关对 hns3 / UB 吞吐差。

KAE 与 DPA： ZIP 在 1–64 KB 的盈亏粒度；低并发（1–8 连接）加密握手延迟；DPA 用户态提交固定开销；引擎间链式传递能力。

UB： 小消息 RTT / Mops / 每 Gbps 核数；RC vs RM 在数千对端下的延迟曲线；CQE 硬件时间戳与 CC 可编程性；Jetty Group 分发策略（有界 / 按方法）；JFCE / 消息中断的通知延迟；Load/Store 远端访存延迟（TP Bypass）。

对标口径： eRPC 2.3 µs / 10 Mrps/ 核；Falcon 单 QP 20 Mops、>100K 连接无悬崖、1 B–2 MB 5.8–8.7 µs；Homa/Linux 18 核 @100 Gbps；HydraRPC 1.47 µs；RPCool 沙箱 0.35 µs；CEIO 零 LLC miss。

顺序的逻辑：1、2 是测量，没有它们后面都是推断；3 是 fleet 平均最大项；4、5、6 是文献已验证、鲲鹏架构位置够用的项；7、8 收益大但依赖调度 / 生态配合。

数据来源与口径：REPORT.md 6.4 节；排序依据 = 文献量级（第一、四部分） × 鲲鹏差距（第 33–35 页） × 可行性。

附录：论文清单（60 篇 + 5 份 slides；[NN] 对应 notes/ 笔记编号）

用户提供（20 篇，笔记 01–19）

Profiling a WSC (ISCA'15) · A Cloud-Scale Characterization of RPCs (SOSP'23) · Lifting the veil on Meta's microservices (ATC'23) · Alibaba Trace Analysis (SoCC'21) · DCPerf (ISCA'25) · DeathStarBench (ASPLOS'19) · CRISP (ATC'22) · Homa (SIGCOMM'18) · R2P2 (ATC'19) · nanoPU (OSDI'21) · RPCValet (ASPLOS'19) · Dagger (ASPLOS'21) · Cerebros (MICRO'21) · HydraRPC (ATC'24) · LightRPC (ICCC'23) · Breakwater (OSDI'20) · Aequitas (SIGCOMM'22) · mRPC (NSDI'23) · zBuffer (PPoPP'26)；另 4 份 slides。

补充：负载刻画（笔记 20–43 之一部分）

Accelerometer (ASPLOS'20) · SoftSKU (ISCA'19) · Profiling Hyperscale Big Data Processing (ISCA'23) · GreenSKU (ISCA'24) · Meta's Hyperscale Infrastructure (CACM'25) · Characterizing a Memory Allocator at WSC (ASPLOS'24) · μManycore (ISCA'23) · Complexity at Scale: Alibaba (arXiv'25) · Alibaba call-graph inconsistencies (ICPE'24) · Host Network Stack Overheads (SIGCOMM'21) · SDT (arXiv'25) · Meta memory subsystem (arXiv'23)

补充：部署形态

ServiceRouter (OSDI'23) · MeshInsight (SoCC'23) · Canal Mesh (SIGCOMM'24) · XLB (arXiv'26) · Snap (SOSP'19) · Service Weaver (arXiv'24) · Junction (NSDI'24) · Lightweight Sidecar for Serverless (SoCC'25)

补充：RPC 通信

eRPC (NSDI'19) · Falcon (SIGCOMM'25，正文 + netdev slides) · Homa/Linux (ATC'21) · RpcNIC (HPCA'25) / RPCAcc (arXiv'24) · RPCool (arXiv'24) · CEIO (SIGCOMM'25) · FlexiNS (arXiv'25) · BenchBF3 (ICNP'24) · BlueField Idiosyncrasies (ICS'25) · NeBuLa (ISCA'20) · Protego (NSDI'23)

补充：序列化 / 压缩

ProtoAcc (MICRO'21) · Optimus Prime (ASPLOS'20) · Cornflakes (SOSP'23) · Zerializer (HotOS'21) · Cereal (ISCA'20) · zFlatBuffers (IPCCC'23) · InNet SerDes (ExaMPI'22) · CDPU (ISCA'23) · SpecProto (ASPLOS'26)

建议的下一步阅读（未下载、被反复引用）

Swift (SIGCOMM'20) · PLB (SIGCOMM'22) · Ultra Ethernet 规范 · XFaaS (SOSP'23) · Shard Manager (SOSP'21) · Naos / Apache Fury （JVM 序列化） · Karandikar 博士论文 (Berkeley 2024) · Intel DSA 文档 · CXL 3.x 多主机共享的真实硬件评估（2025–2026）

数据来源与口径：全部 PDF 在 RPC_PAPERS/ 与 RPC_PAPERS/downloaded/（编号前缀 A 刻画 / B 部署 / C 通信 / D 序列化）；pdftotext 全文在 text/；每篇的结构化笔记在 notes/，总表 notes/INDEX.md。原文图表按 [NN] Fig. N 标注截取自对应 PDF。